

|      |            |
|------|------------|
| 队伍编号 | MC25008384 |
| 题号   | C          |

## 基于随机森林与 STFT 特征分析的自适应音频处理建模

### 摘 要

随着数字音频技术的广泛应用，如何在存储效率、音质保真度与噪声抑制之间实现平衡成为关键挑战。本文围绕音频编码优化与噪声处理两大核心问题，基于数学建模方法提出系统性解决方案。

**对于问题一：**本研究首先构建了一个**多准则综合评价模型**。该模型采用**加权和**方法，整合了三个量化指标：存储效率（基于文件大小比）、音质保真度（基于 **PEAQ 标准** 的客观差异等级 ODG）和编解码复杂度（基于解码时间）。通过设定**场景依赖的权重**，计算对应的**综合评分**。模型结果表明，最优格式选择存在显著的场景依赖性和内容类型差异性。为衡量存储与质量之间的平衡关系，研究引入了**平衡分数（BS）模型**，**BS 曲线**的可视化分析揭示了各格式在效率-质量上的定位。

**对于问题二：**本研究首先提出关于**文件大小的计算模型**。针对 WAV 和 MP3 格式，验证文件大小与采样率、比特深度、声道数和时长成正比。对 AAC 格式，文件大小与之成正相关。随后，采用**双指标评估体系**作为音质评分依据，结合音质保真度和**感知信噪比**。分析音质得分数据证明，更大的采样率、比特深度以及更高码率的压缩算法，音频质量更高。其次，构建了**音频文件性价比指标模型**，由音质评分和文件大小共同决定。该模型得出最佳参数为，针对语音内容，推荐采用 **AAC 压缩算法，采样率 44.1kHz，比特率 96kbps**；针对音乐内容，推荐采用 **MP3 压缩算法，采样率 44.1kHz，比特率 64kbps**。

**对于问题三：**针对传统音频固定参数编码在不同场景下音质与效率难以兼顾且无法动态调整的问题，本文提出一种**多场景自适应编码优化方案**。该方案利用 **YAMNet 模型** 进行音频类型识别，并构建基于性价比的**随机森林回归模型**预测压缩性能。核心在于，系统能根据移动播放、高保真收藏和通用下载三种场景，智能推荐最优压缩参数组合。针对附件 1 中的原始音乐和语音样本，优化后的参数选择在保证音质的同时显著减少了文件大小。例如，在通用下载场景下，针对语音内容，推荐采用 **AAC 压缩算法，采样率 8000Hz，比特率 16kbps**；在高保真收藏场景下，针对音乐内容，推荐采用 **AAC 压缩算法，采样率 48kHz，比特率 140kbps**。与固定参数方案比较时，优化方案在不同场景下均表现出较高的性价比。

**对于问题四：**首先，本研究基于短时傅里叶变换进行时频分析，提取多维声学特征。基于这些特征和预设阈值，建立**规则模型**以识别噪声类型。其次，设计**模块化自适应去噪系统**，根据识别结果级联调用优先级排序的去噪算法。以最大信噪比为目标，通过网格搜索优化去噪算法参数。应用于样本音频，该系统识别出 part1.wav 和 part2.wav 均包含**突发噪声**与**带状噪声**，优化后的音频的信噪比分别为 **18.55 dB** 和 **20.77dB**。该模型适用于中等强度、特征清晰的噪声组合，但在噪声类型复杂多变场景下存在局限性。

**关键词：**PEAQ 感知信噪比 多场景自适应编码 随机森林回归模型 自适应去噪

# 目 录

|                           |    |
|---------------------------|----|
| 摘 要 .....                 | 1  |
| 一、 问题重述 .....             | 1  |
| 二、 问题分析 .....             | 1  |
| 2.1 问题一的分析 .....          | 1  |
| 2.2 问题二的分析 .....          | 1  |
| 2.3 问题三的分析 .....          | 1  |
| 三、 模型假设 .....             | 1  |
| 四、 符号说明 .....             | 2  |
| 五、 问题一的模型建立与求解 .....      | 2  |
| 5.1 综合评价模型建立 .....        | 2  |
| 5.1.1 存储效率 .....          | 2  |
| 5.1.2 音质保真度 .....         | 3  |
| 5.1.3 编解码复杂度 .....        | 4  |
| 5.1.4 不同场景下的权重设定 .....    | 4  |
| 5.1.5 综合评价得分 .....        | 4  |
| 5.2 综合评价模型求解 .....        | 4  |
| 5.2.1 基础指标获取 .....        | 4  |
| 5.2.2 综合评价的结果与分析 .....    | 5  |
| 5.3 平衡分数模型建立 .....        | 6  |
| 5.4 平衡分数模型结果与分析 .....     | 6  |
| 5.5 模型评价 .....            | 7  |
| 5.5.1 模型优势 .....          | 7  |
| 5.5.2 模型局限性 .....         | 7  |
| 六、 问题二的模型建立与求解 .....      | 8  |
| 6.1 参数对文件大小的影响建模 .....    | 8  |
| 6.1.1 WAV 组 .....         | 8  |
| 6.1.2 MP3 组 .....         | 8  |
| 6.1.3 AAC 组 .....         | 8  |
| 6.2 音频信号质量评价指标 .....      | 8  |
| 6.2.1 音频质量的参数选取 .....     | 8  |
| 6.2.2 参数对音频质量的影响 .....    | 9  |
| 6.3 性价比指标建模 .....         | 10 |
| 6.3.1 文件大小比 .....         | 10 |
| 6.3.2 感知信噪比评分 .....       | 10 |
| 6.3.3 性价比计算 .....         | 10 |
| 6.4 模型求解 .....            | 11 |
| 6.4.1 参数对文件大小影响模型求解 ..... | 11 |
| 6.4.2 参数对音频质量影响模型求解 ..... | 12 |
| 6.4.3 性价比指标模型求解 .....     | 13 |
| 6.5 最佳参数推荐结果与分析 .....     | 13 |
| 七、 问题三的模型建立与求解 .....      | 16 |

|                                  |    |
|----------------------------------|----|
| 7.1 多场景自适应音频编码优化方案建立 .....       | 16 |
| 7.1.1 频谱特点和动态范围识别模块设计 .....      | 17 |
| 7.1.2 音频类型识别模块设计 .....           | 17 |
| 7.1.3 压缩参数的场景化推荐设计 .....         | 18 |
| 7.2 多场景自适应音频编码优化方案求解 .....       | 20 |
| 7.2.1 模型数据集构建 .....              | 20 |
| 7.2.2 超参数调优 .....                | 20 |
| 7.2.3 多场景最佳参数优化方案求解 .....        | 20 |
| 7.2.3 多场景最佳参数优化方案与固定参数方案对比 ..... | 22 |
| 7.3 多场景自适应音频编码优化方案评价 .....       | 22 |
| 7.3.1 模型指标评估 .....               | 22 |
| 7.3.2 方案评价和改进建议 .....            | 23 |
| 八、 问题四的模型建立与求解 .....             | 23 |
| 8.1 噪声特征提取与选择 .....              | 23 |
| 8.1.1 噪声类型特征分析 .....             | 23 |
| 8.1.2 噪声特征参数选取 .....             | 23 |
| 8.2 噪声识别模型的建立 .....              | 24 |
| 8.2.1 输入向量 $X$ .....             | 24 |
| 8.2.2 模型参数 $V$ .....             | 24 |
| 8.2.3 决策规则 .....                 | 25 |
| 8.2.4 输出结果 .....                 | 25 |
| 8.3 自适应去噪模型建立 .....              | 25 |
| 8.3.1 噪声识别模块 .....               | 25 |
| 8.3.2 降噪策略模块 .....               | 25 |
| 8.3.3 自适应参数调节模块 .....            | 26 |
| 8.4 噪声识别模型求解 .....               | 26 |
| 8.4.1 时频分析及噪声特征参数 .....          | 26 |
| 8.4.2 噪识别模型求解 .....              | 26 |
| 8.4.3 自适应去噪模型求解及结论 .....         | 27 |
| 8.5 模型评价 .....                   | 28 |
| 8.5.1 模型优势 .....                 | 28 |
| 8.5.2 模型局限性 .....                | 28 |
| 参考文献 .....                       | 29 |
| 附录 .....                         | 30 |

## 一、问题重述

**问题一：**构建综合评价指标的模型结构，拟合存储效率与音质保真度之间平衡关系的量化方式，根据该方式计算不同音频在各种使用场景下的得分，为不同场景推荐最优格式。

**问题二：**分析音频文件在不同采样率、比特深度和压缩算法组合下的音质与文件大小之间的关系。引入合理的评价指标对“音质”进行量化，同时考虑文件大小带来的存储或传输成本，设计一个反映音质与体积之间权衡关系的性价比指标。

**问题三：**旨在设计一种自适应音频编码方案，能够根据输入音频的特征自动选择最优的编码参数，在保证音质保真的前提下实现文件大小的最小化。

**问题四：**要求对附件 2 中的音频样本进行时频域分析，识别并量化音频中的多种噪声类型，并基于分析结果构建数学模型，提取各类噪声的特征参数。在此基础上，需设计一种改进的自适应去噪算法，能够根据噪声类型和强度自动选择最合适的去噪策略。

## 二、问题分析

### 2.1 问题一的分析

问题一旨在设计综合评价指标，量化 WAV、MP3、AAC 等音频格式在存储与音质间的平衡。关键在于整合文件大小、音质损失、计算复杂度及适用场景等冲突因素，构建数学模型并运用多准则决策方法进行归一化融合，最终得出反映特定场景下格式综合性能的评分。

### 2.2 问题二的分析

问题二旨在建立数学模型分析采样率、比特深度及压缩算法对音频质量与文件大小的影响。核心任务是设计一个性价比指标，用以平衡音质和存储占用，并据此对附件 1 中的文件进行排序，最终为语音和音乐内容分别推荐最优参数组合。

### 2.3 问题三的分析

问题三要求设计自适应音频编码方案。关键在于通过分析输入音频特征（如类型、频谱、动态范围）建立模型，以自动选择最优编码参数。需将此方案应用于原始样本，并与固定参数编码对比，量化展示其在文件大小及音质保真度上的改进效果。

### 2.4 问题四的分析

问题四聚焦于音频去噪。要求运用时频分析建立数学模型，以识别并量化样本音频中的各类噪声特征。核心是提出一种能根据噪声类型自适应选择处理方法的改进去噪策略，并应用于样本，需报告识别出的噪声种类、去噪后的信噪比及分析其适用性与局限。

## 三、模型假设

**假设一：**参数独立性。假设采样率、比特深度、压缩算法等参数对文件大小和音质的影响相互独立，可分别建模分析，且不同参数组合的影响可通过线性加权综合评估。

**假设二：**处理流程独立性假设。假设音频预处理不造成额外失真，且模型推理与分类过程对输入音频的结构特征保持不变，确保特征提取的准确性。

**假设三：**噪声类型界定清晰。假设音频中存在的噪声可以明确且完全地归属于背景

噪声、突发噪声、带状噪声这三类之一或其组合。

**假设四：**去噪算法针对性有效。假设选用的去噪处理仅作用于对应模块，且不会对处理的音频引入新的噪声。

**假设五：**信号与噪声可分。假设目标信号与噪声信号在经过适当变换后是线性可分的，或者至少可以通过算法操作分离。

## 四、符号说明

| 符号       | 含义                  | 符号              | 含义                      |
|----------|---------------------|-----------------|-------------------------|
| $CS$     | 音频格式的综合评价得分         | $BR$            | 比特率                     |
| $Se$     | 音频格式的存储效率           | $CR$            | 压缩比                     |
| $Q$      | 音频格式的音质保真度          | $SNR$           | 信噪比                     |
| $Sc$     | 编解码复杂度（计算资源消耗）      | $P_{signal}$    | 原始信号功率                  |
| $DV$     | 文件大小（存储空间占用）        | $P_{noisc}$     | 失真或噪声的功率                |
| $t$      | CPU 解码音频文件加载到内存中的时长 | $SNR_p$         | 感知信噪比                   |
| $Ref$    | 文件大小比               | $MSE$           | LATS 中的均误方差             |
| $ODG$    | PEAQ 方法下的音质得分       | $N$             | 频率点总数                   |
| $w_i$    | 权重                  | $S_i$           | 样本音频第 $i$ 个频率点上的 LATS 值 |
| $BS$     | 平衡分数                | $R_i$           | 参考音频第 $i$ 个频率点上的 LATS 值 |
| $\alpha$ | 平衡权重因子              | $PC$            | 感知信噪比评分                 |
| $SR$     | 采样率                 | $CPR$           | 音频的性价比                  |
| $BD$     | 比特深度                | $C$             | 声道数                     |
| $X$      | 噪声的特征参数向量           | $T$             | 音频时长                    |
| $x_i$    | 第 $i$ 个噪声特征参数       | $W$             | 噪声类型向量                  |
| $V$      | 特征参数阈值向量            | $Y_i, Y_j, Y_k$ | 指示噪声类型的参数               |
| $t_i$    | 第 $i$ 个噪声特征参数阈值     |                 |                         |

## 五、问题一的模型建立与求解

### 5.1 综合评价模型建立

考虑存储效率、音质保真度以及编解码复杂度等作为影响因素，考虑不同场景的偏好赋予影响因素不同的权重，建立综合评价模型。引入综合分数  $CS$  作为选择音频格式的决定指标。

#### 5.1.1 存储效率

在确定音频采样率的前提下，对音频进行分组对比处理，分别到的音乐与语音两组的存储效率  $Se$ 。

##### （1）标量化文件大小比

一段音频存储效率  $Se$  的计算方式为计算其压缩率，由于音频（经过压缩存储后的音频）的文件大小  $DV$  与母音频（原始参考音频）的文件大小  $DV$  作比得到其文件大小比

Ref。计算公式为

$$Ref = \frac{DV_{current}}{DV_{init}}$$

### （2）文件大小比归一化

一段音频的存储效率应与其文件大小比 Ref 呈负相关。

在获得每组音频与母音频的文件大小比 Ref 后，为了便于在后续的计算或模型中作为参数使用，需要对这些文件大小比进行归一化处理。归一化的目的是将不同范围的文件大小比映射到一个标准区间[0, 1]内，这样可以消除不同量级数据的影响，提升数据的可比性和模型训练的稳定性等。

我们采用  $1 - Ref$  的方式对数据集进行归一化。其原理是利用文件大小比 Ref 与存储效率的负相关特性，通过用 1 减去原始文件大小比 Ref，将其映射到[0, 1]的标准区间。计算公式为：

$$Se = 1 - Ref$$

经过归一化处理，数值越大表示存储效率越高，数值越小表示存储效率越低，有效地体现音频存储效率。

## 5.1.2 音质保真度

本文采用了国际电信联盟（ITU）推荐的客观音频质量评估标准——PEAQ，以此量化音频在编码或压缩过程中的质量损失，计算不同音频格式对应的音质保真度 $Q$ 。

### （1）音质客观评估方法与参数选取

PEAQ 算法基于人类听觉系统的感知机制设计，模拟听觉系统在实际听音过程中的主观判断，从而实现对音频质量的客观评分。该方法依赖于原始音频与测试音频之间的差异进行建模，其核心目标是通过信号处理与心理声学建模技术，评估音频处理对听觉感知质量的影响。

PEAQ 算法主要由前端处理模块和后端评分模块两部分组成：

（i）前端模块：该模块对输入音频信号进行预处理，随后，从音频中提取一系列与人耳听觉感知密切相关的低层感知特征。

（ii）后端评分模块：根据提取的感知特征，结合心理声学模型，计算参考音频与测试音频之间的感知距离，并利用训练好的神经网络或多项式回归模型将 MOVs 映射到一个最终的主观评分指标——ODG。取值于区间[-4,0]，其中 0 代表无可察觉失真，-4 代表严重失真。

### （2）基于 PEAQ 的音频保真度评估流程

首先，进行参数一致性校验。为确保 PEAQ 评分的准确性，所有音频对需具备相同的采样率、位深度和声道结构，与原始音频不一致的，需要进行裁剪。

然后是特征提取与评分计算输入音频对。向 PEAQ 系统输入音频对，输出每段测试音频对应的 ODG 值。该值作为音质损失程度的度量指标，反映了处理过程对原始音质的影响。

### （3）ODG 值归一化

由于音质保真度与音质损失呈负相关，因此对 ODG 值进行归一化，计算公式为：

$$Q = \frac{ODG + 4}{4}$$

在此映射下，当  $ODG = 0$  时， $Q = 1$  表示无损保真；当  $ODG = -4$  时， $Q = 0$  表示音质最差。

### 5.1.3 编解码复杂度

编解码复杂度反映了对音频进行编码和解码操作的难易程度和计算资源消耗情况。因此，以 CPU 解码音频文件加载到内存中的时长  $t$  为指标量化音频的复杂程度， $t$  越小，表示所需的计算资源消耗越少，编解码复杂度越低。

数据进行归一化处理。采用基于 Min - Max 归一化法，将编解码时间映射到  $[0, 1]$  区间，音频编解码复杂度越高，其对应得分应越低。公式为

$$Sc = 1 - \frac{t - t_{min}}{t_{max} - t_{min}}$$

### 5.1.4 不同场景下的权重设定

(1) 权重定义与场景偏好

通过设置不同的权重来反应不同场景对音频特性的偏好和需求。设定三个权重  $w_1, w_2, w_3$ ，分别对应存储效率、音质保真度和解码复杂度。权重之间满足以下约束条件：

$$w_1 \geq 0, w_2 \geq 0, w_3 \geq 0 \quad (1)$$

$$w_1 + w_2 + w_3 = 1 \quad (2)$$

(2) 权重设定

考虑三种不同的音频使用场景设置了以下权重设定

表 1.1 不同场景下权重设定

| 场景                | $w_1$ | $w_2$ | $w_3$ |
|-------------------|-------|-------|-------|
| 移动流媒体（偏重存储，兼顾流畅度） | 0.45  | 0.25  | 0.3   |
| 高保真音乐收藏（极度偏重音质）   | 0.1   | 0.8   | 0.1   |
| 通用下载/存储（均衡考虑）     | 0.35  | 0.35  | 0.3   |

### 5.1.5 综合评价得分

综合三种影响因素和场景不同加权得到综合评价得分，作为选择音频格式的重要依据，计算公式如下：

$$CS = w_1 * Se + w_2 * Q + w_3 * Sc$$

## 5.2 综合评价模型求解

### 5.2.1 基础指标获取

将附件一的音频样本分为音乐组和语音组，只采用 44100 Hz 采样率的数据进行统计。分别选取 语音\_44100Hz\_16bit.wav 和 音乐\_44100Hz\_16bit.wav 作为原始参考音频，统计其他子音频的音频特征。

(1) 语音组

表 1.2 语音组音频特征

| 文件名                        | 存储效率 Se     | 音质保真度 Q | 编解码复杂度<br>评分 Sc |
|----------------------------|-------------|---------|-----------------|
| 语音_44100Hz_16bit.wav       | 0           | 1       | 1               |
| 语音_44100Hz_MP3_320kbps.mp3 | 0.541371096 | 0.7125  | 0.174           |
| ...                        | ...         | ...     | ...             |

|                           |             |       |        |
|---------------------------|-------------|-------|--------|
| 语音_44100Hz_AAC_96kbps.aac | 0.935446121 | 0.624 | 0.0914 |
|---------------------------|-------------|-------|--------|

(2) 音乐组

表 1.3 音乐组音频特征

| 文件名                        | 存储效率 Se     | 音质保真度 Q | 编解码复杂度评分 Sc |
|----------------------------|-------------|---------|-------------|
| 音乐_44100Hz_16bit.wav       | 0           | 1       | 1           |
| 音乐_44100Hz_MP3_320kbps.mp3 | 0.54386856  | 0.98    | 0.9999      |
| ...                        | ...         | ...     | ...         |
| 音乐_44100Hz_AAC_128kbps.aac | 0.810410819 | 0.9335  | 0.1832      |

5.2.2 综合评价的结果与分析

根据不同的场景赋予影响因素对应的权重，计算得到各个音频在不同场景下的综合评分，结果如图一和图二所示。通过对图像的分析，可以得出以下结论：

显著的场景依赖性，没有任何一种音频格式在所有场景下都表现最优。不同格式的综合评分随应用场景的变化而显著改变，这验证了综合评价模型中引入场景权重区分的必要性和有效性。

内容类型差异性。对比音乐和语音两张热力图，可以看出格式的相对表现存在差异。AAC 格式在语音内容的”移动流媒体”和”通用下载/存储”场景下，相较于 MP3 在同等比特率下似乎更具优势，这可能与编码器针对不同内容类型的优化以及人耳感知的差异有关。

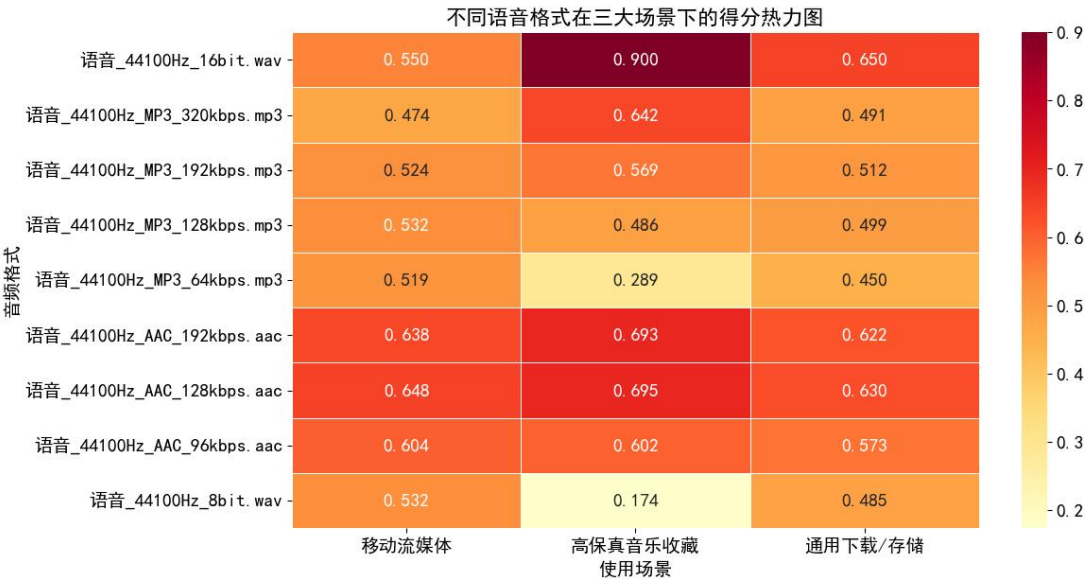


图 1.1 语音组综合评分热力图



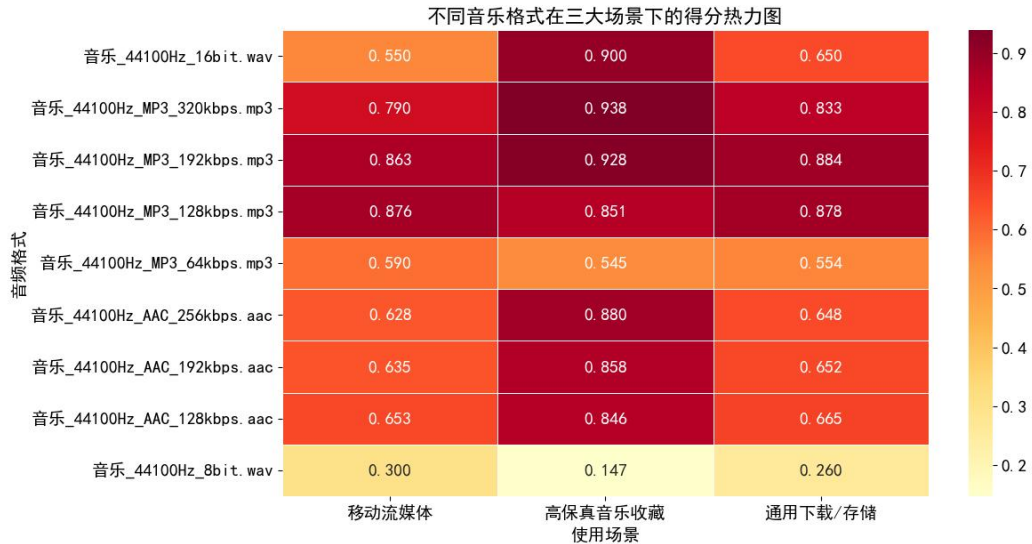


图 1.2 音乐组综合评分热力图

### 5.3 平衡分数模型建立

通常，提高存储效会伴随音质的损失，反之亦然。为了定量地分析和比较不同音频格式在处理这一核心矛盾上的表现，并识别在不同偏好侧重下表现更优的格式，本研究引入了平衡分数模型。

区别于旨在提供全面评估的综合评价模型，平衡分数模型特别专注于量化和分析存储效率与音质保真度这两个关键维度之间的内在平衡与矛盾关系。利用 BS 模型，识别出那些在 Se-Q 平面上表现最优的音频格式。这些格式代表了在给定偏好下，存储效率和音质保真度所能达到的最佳平衡点，它们通常位于或接近所谓的帕累托有效前沿，即所有最优 Se-Q 组合的集合。

根据上述目的和分析定义平衡分数公式如下：

$$BS = \alpha * Se + (1 - \alpha) * Q \quad \alpha \in [0,1]$$

平衡权重因子 $\alpha$ 用于表示用户或特定应用场景对存储效率和音质保真度的相对偏好程度。当 $\alpha$ 趋近于 1 时，表示更侧重于存储效率。当 $\alpha$ 趋近于 0 时，表示更侧重于音质保真度。

### 5.4 平衡分数模型结果与分析

平衡分数模型使用的数据集沿用综合评价模型中得到数据集，并根据数据集绘制图像并进行分析

$\alpha$ 以 0.1 为步长，绘制不同平衡权重下的音频数据的平衡分数曲线，结果如图 3，4。

该 BS 曲线为在特定需求下寻找最佳平衡点的音频格式提供了直接的决策依据，并清晰地展示了不同格式在整个效率-质量偏好谱系上的适用区间和相对优劣。

为了直观展示每种音频格式在存储效率和音质保真度之间的静态表现，绘制存储率与音质保真度之间的平衡图，如图 5，6 所示。

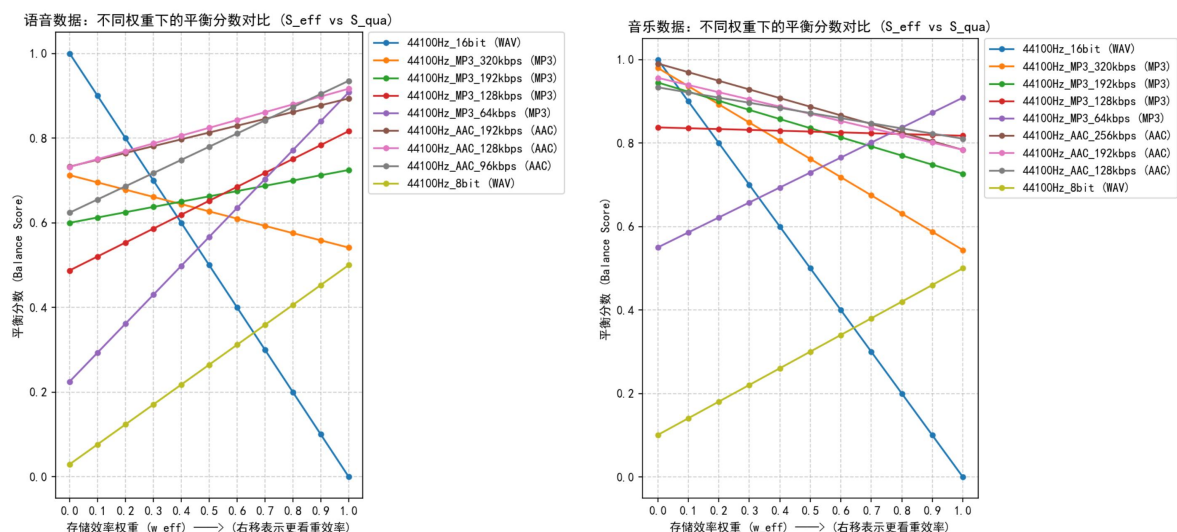


图 1.3 图 1.4 语音和音乐数据在不同权重下的平衡分数对比

Se-Q 散点图中各点的位置反映了不同格式的固有特性。图中靠右上方边界的点 6,7,8 构成了一个近似的帕累托有效前沿。任何远离此边界的点，理论上都存在更优的平衡选择。

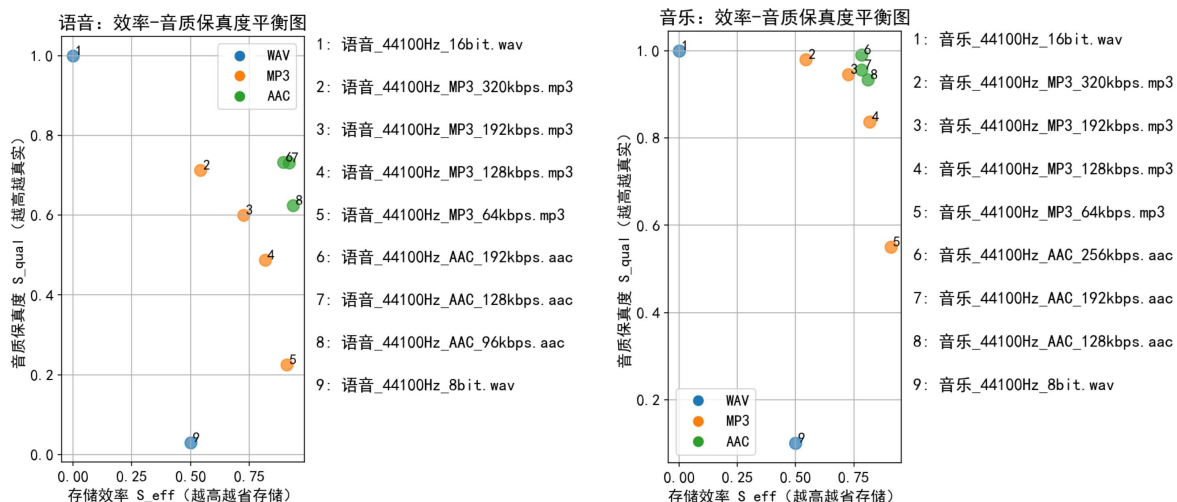


图 1.5 图 1.6 语音和音乐的效率-音质保真度平衡图

对比两张图，说明存储效率和音质保真度的具体平衡关系受到音频内容类型的影响。

## 5.5 模型评价

### 5.5.1 模型优势

综合评价模型通过引入场景权重机制与多指标归一化方法，有效实现了存储效率、音质保真度及编解码复杂度的动态平衡，具有以下优势：

(1) 场景适应性，通过权重向量灵活调整不同场景的偏好。模型输出的热力图显示，不同音频格式的综合评分随场景变化显著，验证了模型对不同需求的精准适配能力。

(2) 帕累托前沿分析，通过平衡分数模型与 Se-Q 散点图，直观展示了存储效率与音质保真度的权衡关系，并通过分析帕累托前沿的方式，为决策提供明确依据。

(3) 无绝对最优解验证：热力图显示“没有一种格式在所有场景最优”，验证了场景权重区分的必要性，同时反映模型对内容类型差异的敏感性。

### 5.5.2 模型局限性

(1) 权重主观性，场景权重依赖人工预设，可能无法完全反映用户实际偏好。

(2) 编解码复杂度指标单一，当前模型仅以 CPU 解码时间衡量复杂度，未考虑内存占用、并行计算能力等因素。

(3) 未考虑实时性与硬件适配，模型未涉不同硬件平台对计算资源的适应性。例如，低功耗设备可能需要更低复杂度的编解码算法，但模型仅通过时长指标间接反映，未建立硬件参数与复杂度的映射关系。

## 六、问题二的模型建立与求解

### 6.1 参数对文件大小的影响建模

为了系统分析不同采样率、比特深度及压缩算法等参数对音频文件大小的影响，我们首先按压缩算法对音频样本进行分类，分别探讨各类格式在不同参数配置下的表现差异。

#### 6.1.1 WAV 组

通过查阅资料以及问题一的模型分析基础，确定 WAV 音频格式的文件大小与采样率 (SR)、比特深度 (BD)、声道数 (C) 以及音频时长 (T) 呈线性正相关。假设如下理论模型：

$$DV_{WAV} = CR * \frac{SR * BD * C * T}{8}$$

说明：

C：声道数，默认为 1。

SR \* BD：每秒钟单个声道产生的比特数。

SR \* BD \* C：每秒钟所有声道产生的总比特数。

SR \* BD \* C \* T：音频总时长内的总比特数。

当音频样本同为 WAV 格式时，理论上对于每一个音频样本，将其各参数代入模型后反推出的系数 CR 应相等。

#### 6.1.2 MP3 组

MP3 是一种广泛使用的有损音频压缩格式，其核心特性在于通过去除人耳不敏感的音频信息，以较小的文件大小保留尽可能高的音质。与无损的 WAV 格式不同，MP3 的文件大小主要由压缩比率和比特率决定。

根据查阅资料与问题一模型的分析基础，MP3 音频格式的文件大小与压缩比率 (CR)、比特率 (BR)、声道数 (C) 以及音频时长 (T) 呈正相关。建立如下理论模型：

$$DV_{MP3} = CR * \frac{BR * C * T}{8}$$

当音频样本同为 MP3 格式，于理论上每一个音频样本，将其各参数代入模型后反推出的系数 CR 应相等。

#### 6.1.3 AAC 组

AAC 是通过 MPEG - 4 标准定义的有损音频压缩格式，属于感知编码体系，依据内容复杂度、压缩策略和编码参数动态生成。由于 AAC 与 MP3 同为有损压缩算法，所以参数对 AAC 音频格式文件大小的建模思路与 MP3 组一致。

### 6.2 音频信号质量评价指标

#### 6.2.1 音频质量的参数选取

在音频压缩与编码参数选择过程中，如何量化压缩对音频质量的影响是一个核心问题。为了全面分析音频编码参数对音频质量的影响，有必要引入一个能够客观反映听觉质量变化的指标。本文选取了感知信噪比与音频格式的音质保真度作为主要的质量评价

指标，最终的评分由二者加权得出。

其中，音频格式的音质保真度  $Q$  的计算方法已在第 5.1.2 节中详细介绍，此处不再赘述。ODG 能有效刻画压缩处理对人耳感知质量的影响。

信噪比作为衡量音频信号质量的重要客观指标，定义为有用信号功率与噪声功率之比，能够直观反映音频信号受到噪声干扰的程度。然而，传统信噪比未能充分考虑人耳对不同频率的敏感性及听觉掩蔽效应，因而在音频质量评价中存在一定局限性。

感知信噪比  $SNR_p$  作为评价音频质量的指标之一，融合了人类听觉特性，在保持物理信噪比精度的基础上，增强了对主观听感一致性的刻画能力。与传统信噪比相比，感知信噪比具有考虑频谱差异、可区分轻微噪声和严重失真、有效处理复杂信号等优势。

感知信噪比对音频进行如下处理：

(1) 时间对齐

将参考音频和测试音频裁剪为相同长度，避免比较偏移或额外补零造成误差。

(2) 短时傅里叶变换

人耳对能量更为敏感。将时域的音频信号转换到频域，得到信号在不同时刻的频谱信息。通过这种转换了解信号在各个频率上的能量分布情况，更符合人耳的感知机制。

(3) 计算频谱幅度

将频谱的能量（或幅度）在时间轴上取平均，得到一个单一的频谱曲线，表示该文件在整个时长内的平均频率能量分布。

经过以上音频处理方式，计算音频的感知信噪比。感知信噪比计算公式如下：

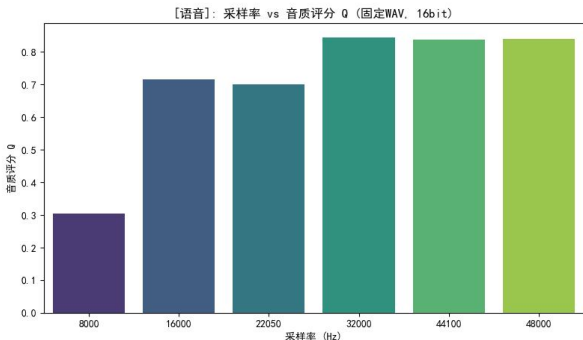
$$SNR_p = 10 \log_{10} \left( \frac{P_{signal}}{P_{noisc}} \right)$$

计算得到感知信噪比，其值越高，表明音频质量越佳；反之，则说明信号受损或失真较为严重。

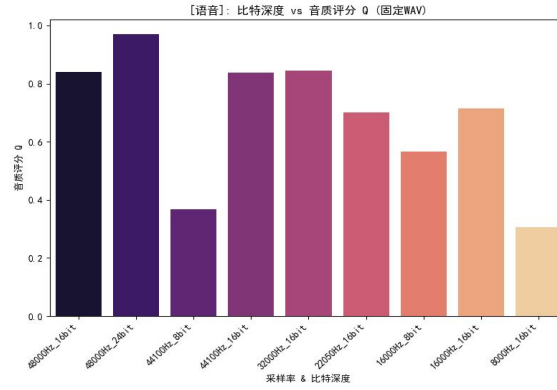
## 6.2.2 参数对音频质量的影响

为定性分析采样率比特深度、压缩算法等参数对音频质量的影响，对比音频样本的采样率大小与其感知信噪比得分高低的关系，以语音样本为例，将结果可视化。

表 2.1 各参数对音频质量影响可视化

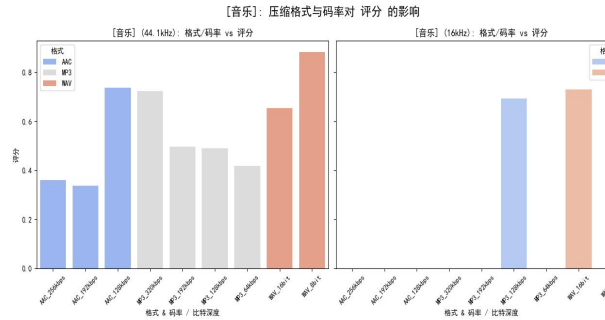
| 影响参数 | 可视化结果                                                                                | 说明                 |
|------|--------------------------------------------------------------------------------------|--------------------|
| 采样率  |  | 音频样本的质量与采样率呈正相关关系。 |

比特深度



当采样率一致时，比特深度高的音频样本具有更高的音频质量。

压缩算法



当采样率一致时，WAV 格式的评分较 AAC 和 MP3 格式更高，说明无损压缩的 WAV 格式在音频质量上整体优于有损压缩的 AAC 和 MP3 格式。

通过定性分析，发现音频质量与采样率、比特深度、压缩算法等参数呈正相关。

## 6.3 性价比指标建模

在音频压缩策略选择中，既要关注音质的保持，又需考虑文件体积的控制，因此有必要引入一个综合衡量压缩效果优劣的指标——性价比。该指标旨在评价每组音频编码参数在保真度与压缩率之间的平衡程度，从而为不同类型内容推荐最优参数组合。

### 6.3.1 文件大小比

如第 5.1.1 节所述，文件大小比用于衡量压缩后音频文件相较于原始文件的体积缩减程度，其数值越小，表示压缩效果越明显。在本节的性价比模型中，文件大小比作为衡量压缩效率的关键因子之一，参与最终的性价比评分计算。

### 6.3.2 感知信噪比评分

为消除不同音频样本间感知信噪比取值差异带来的影响，需对 PSNR 进行归一化处理，得到感知信噪比评分：

$$PC = \frac{SNR_p - SNR_{pmin}}{SNR_{pmax} - SNR_{pmin}}$$

其中，PSNR 表示每个音频样本计算所得的感知信噪比，PC 的取值范围归一化至 [0, 1] 区间，数值越大代表音频质量越高。

### 6.3.3 性价比计算

综合考虑音质与文件大小，通过二者的量化，定义音频的性价比计算公式：

$$CPR = \frac{PC * w_1 + Q * w_2}{Ref}$$

其中，PC 表示感知信噪比评分，Q 表示音频格式的音质保真度， $w_1$  和  $w_2$  分别表示赋予的权重，Ref 表示文件大小比。性价比指标数值越高，表明在保持较好音质的同时，实现了更高效的压缩率。

考虑到 PEAQ 比 SNR 更能反映“听起来怎么样”。因此，给  $w_2$  权重 0.7， $w_1$  权重 0.3 根据上述指标，对每个音频样本计算其性价比得分，并对其结果进行排序。进一步，分别对语音类与音乐类音频样本独立排序分析，从中选出最优的编码参数组合，作为推荐方案。



## 6.4 模型求解

### 6.4.1 参数对文件大小影响模型求解

#### (1) WAV 组

反推系数公式：

$$CR = DV_{WAV} * \frac{8}{SR * BD * C * T}$$

根据 6.1.1 中 WAV 格式文件大小的计算公式及参数值，得到各中样本中系数 CR 的大小。

表 2.2 WAV 格式样本各参数值

| 文件名                  | 采样率/Hz | 比特深度/bits | 音频时长/s | 文件大小/bytes | CR          |
|----------------------|--------|-----------|--------|------------|-------------|
| 语音_48000Hz_24bit.wav | 48000  | 24        | 10     | 859370     | 1.000030556 |
| 语音_48000Hz_16bit.wav | 48000  | 16        | 5.97   | 572928     | 0.999639401 |
| ...                  | ...    | ...       | ...    | ...        | ...         |
| 音乐_16000Hz_16bit.wav | 16000  | 16        | 10     | 320044     | 1.0001375   |
| 音乐_8000Hz_16bit.wav  | 8000   | 16        | 10     | 160044     | 1.000275    |

实验发现每一个样本中 CR 近乎为 1，与模型推测一致。

故 WAV 格式文件大小与各参数满足：

$$DV_{WAV} = \frac{SR * BD * C * T}{8}$$

#### (2) MP3 组

反推系数公式：

$$CR = DV_{MP3} * \frac{8}{BR * C * T}$$

根据 6.1.2 中 MP3 格式文件大小的计算公式及参数值，得到各中样本中系数 CR 的大小。

表 2.3 MP3 格式样本各参数值

| 文件名                        | 采样率/Hz | 比特率/kbps | 音频时长/s | 文件大小/bytes | CR          |
|----------------------------|--------|----------|--------|------------|-------------|
| 语音_44100Hz_MP3_320kbps.mp3 | 44100  | 320      | 5.97   | 241414     | 1.010946399 |
| 语音_44100Hz_MP3_192kbps.mp3 | 44100  | 192      | 5.97   | 144865     | 1.011062256 |
| ...                        | ...    | ...      | ...    | ...        | ...         |
| 音乐_44100Hz_MP3_64kbps.mp3  | 44100  | 64       | 10     | 80500      | 1.00625     |
| 音乐_16000Hz_MP3_128kbps.mp3 | 16000  | 128      | 10     | 161900     | 1.011875    |

实验发现每一个样本中 CR 近乎为 1.01，与推测一致，对比 WAV 的 CR 近乎为 1，查阅资料知 mp3 的压缩算法为有损压缩，故 CR 不一致。

故 MP3 格式文件大小与各参数满足：

$$DV_{MP3} = \frac{BR * C * T}{8}$$

(3) AAC 与 MP3 同为有损压缩算法，经过与 6.1.2 的假设和 6.4.1 (2) 相同的计算及验证后，发现 AAC 格式音频样本的文件大小与公式的拟合程度低。

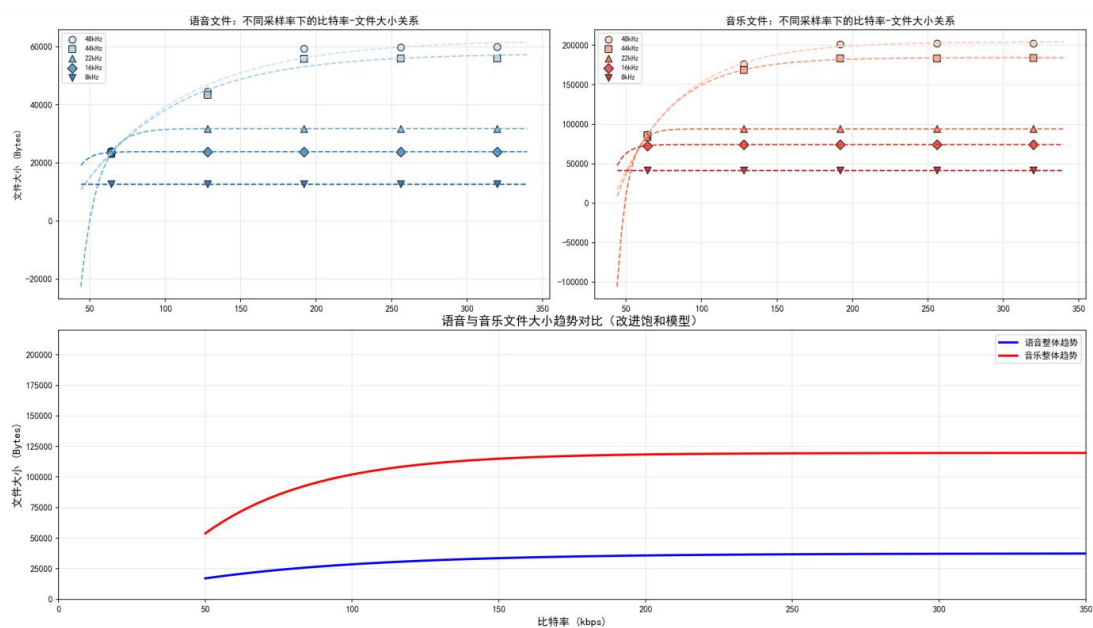


图 2.1 AAC 格式的文件大小与比特率的拟合关系图

实验结果显示，不同 AAC 文件的压缩调整系数 CR 存在一定波动，主要原因为 AAC 文件大小差异主要源于其复杂的动态压缩机制、结构性开销及可变比特率在复杂片段中的自适应调整。

经过查阅资料，AAC 文件格式编码方式复杂，存在多种影响因素，具有编码参数多样，压缩算法复杂且存在可变比特率等特点，其文件大小无法利用固定公式计算。通过定性分析与拟合验证，其文件大小与比特率呈正相关。

综上，虽然存在一定误差，AAC 格式音频的文件大小在整体上仍可近似看作与比特率、声道数和音频时长成正比。其经验模型可表示为：

$$DV_{AAC} \propto BR * C * T$$

#### 6.4.2 参数对音频质量影响模型求解

经过对音频的处理，将样本分为音乐组与语音组，由公式计算音频样本的感知信噪比和 PEAQ 评分。经过计算得到不同采样率、比特率及格式下语音音频样本的感知信噪比和 PEAQ 评分。将两组感知信噪比和 PEAQ 评分结果可视化：

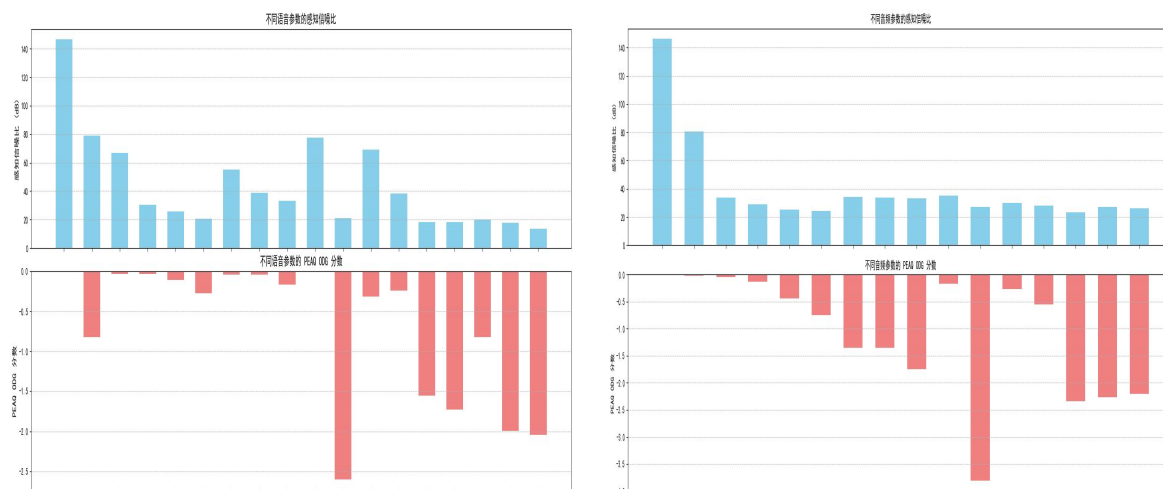


图 2.2 图 2.3 不同语音和音乐参数的感知信噪比与 ODG 分数  
语音组中，“语音\_48000Hz\_24bit.wav” 感知信噪比最大，柱形显著高于其他文件，

ODG 分数为零，说明其音频质量最高；”语音\_16000Hz\_8bit.wav” 等文件感知信噪比最小，柱形低矮，ODG 柱形分数较高，说明其音频质量较低。

音乐组中，”音乐\_48000Hz\_24bit.wav” 在图表中感知信噪比柱形最高，ODG 分数柱形最低，表明该音频样本信号音频质量最高；”音乐\_44100Hz\_80bit.wav” 等部分文件感知信噪比相对较小，ODG 分数柱形较高，意味着该音频样品音频质量较差。

6.4.3 性价比指标模型求解

根据 6.3 节所构建的性价比计算模型，本文对所有音频样本进行归一化处理与性价比评分，统计结果如表所示，分为语音组与音乐组两类。

(1) 语音组样本结果：

表 2.4 语音组样本各指标

| 文件名                        | 文件大小比       | 音质评分     | 性价比         |
|----------------------------|-------------|----------|-------------|
| 语音_44100Hz_AAC_96kbps.aac  | 0.039540594 | 0.97095  | 9.88414875  |
| 语音_44100Hz_AAC_128kbps.aac | 0.051019933 | 0.840087 | 8.421038142 |
| ...                        | ...         | ...      | ...         |
| 语音_44100Hz_MP3_64kbps.mp3  | 0.056223745 | 0.690401 | 5.660040503 |
| 语音_16000Hz_MP3_64kbps.mp3  | 0.056688039 | 0.425743 | 3.432216815 |
| ...                        | ...         | ...      | ...         |
| 语音_44100Hz_MP3_320kbps.mp3 | 0.280919744 | 0.838433 | 2.55925085  |
| 语音_44100Hz_8bit.wav        | 0.306286    | 0.304814 | 0.476138672 |

(2) 音乐组样本结果：

表 2.5 音乐组样本各指标

| 文件名                        | 文件大小比    | 音质评分        | 性价比      |
|----------------------------|----------|-------------|----------|
| 音乐_44100Hz_MP3_64kbps.mp3  | 0.055901 | 0.868852307 | 10.61736 |
| 音乐_44100Hz_MP3_128kbps.mp3 | 0.111772 | 0.774688139 | 5.812852 |
| ...                        | ...      | ...         | ...      |
| 音乐_44100Hz_AAC_256kbps.aac | 0.132516 | 0.497245617 | 3.841804 |
| 音乐_44100Hz_AAC_192kbps.aac | 0.132415 | 0.48924069  | 3.840598 |
| ...                        | ...      | ...         | ...      |
| 音乐_48000Hz_24bit.wav       | 1        | 0.731194756 | 0.999752 |
| 音乐_44100Hz_8bit.wav        | 0.306271 | 0.062433889 | 0.157455 |

6.5 最佳参数推荐结果与分析

为便于直观比较，本文对不同音频参数组合的性价比进行排序与可视化，分别得到语音类与音乐类音频中的最优编码方案。

(1) 语音类音频推荐结果：



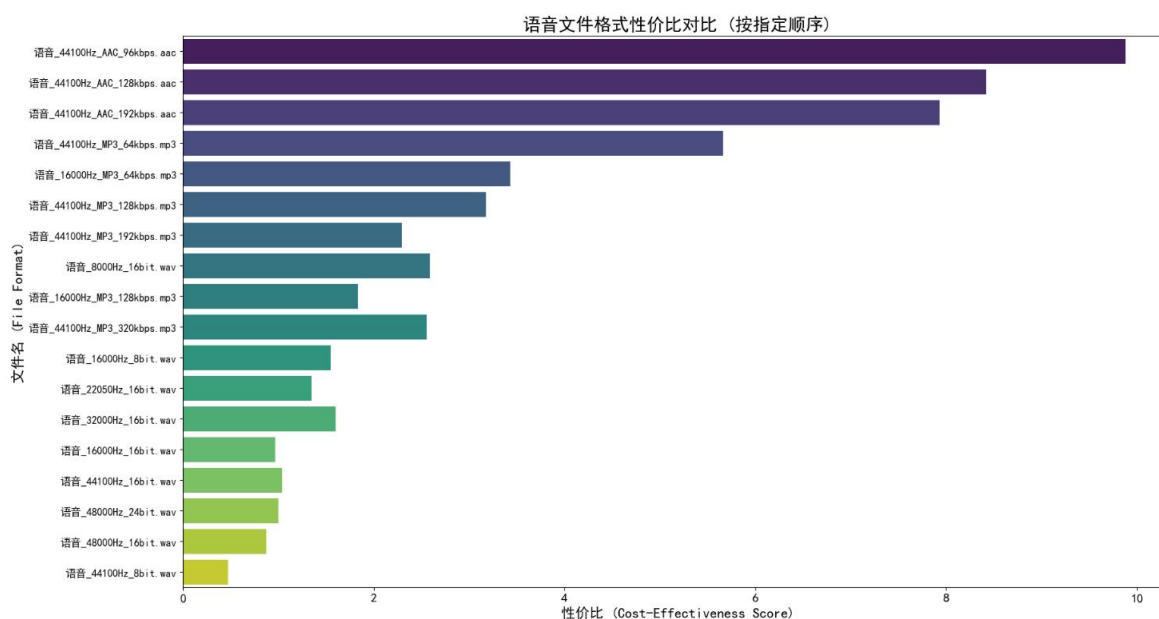


图 2.4 语音文件格式性价比对比

综合性价比排序，”语音\_44100Hz\_AAC\_96kbps.aac” 样本得分最高，表明其在压缩效率与音质之间达到了最优平衡。因此，推荐语音内容压缩参数如下

采样率：44100Hz，比特深度：96kbps，压缩算法：AAC

(2) 音乐类音频推荐结果：

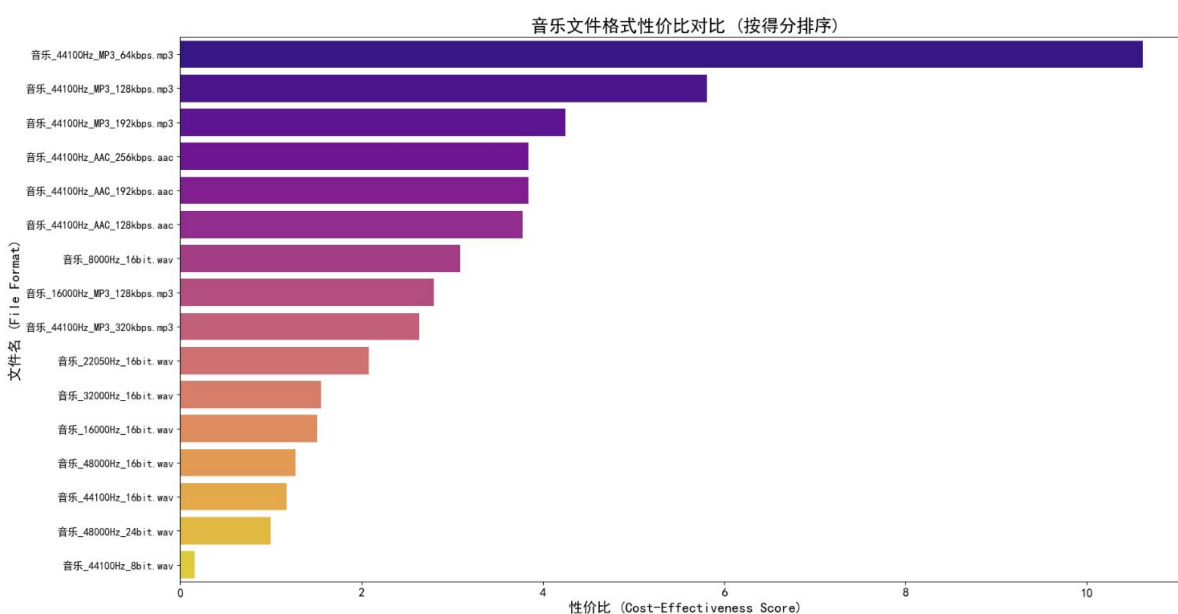


图 2.5 音乐文件格式性价比对比

如图，音乐组中性价比最高的音频样本为”音乐\_44100Hz\_MP3\_64kbps.mp3”，故语音内容中推荐到最佳参数为：

采样率：44100Hz，比特深度：64kbps，压缩算法：MP3

## 6.6 模型准确性与评价

模型中引入感知信噪比作为评估音频质量的指标。相对于参考音频，音频样本某些频段上出现截断、失真、能量损失或是其他音质损失都会导致其感知信噪比得分降低。引入 LATS（长时平均频谱），可视化音频样本与参考音频的以上音频特征进行对比，结合样本的感知信噪比得分来验证模型准确性。二者的结合能够实现从物理能量分布到主观感知质量的双维度验证。

(1) LATS 分析原理

LTAS 是将音频样本在整个时长内的频谱能量进行时间平均，生成一条平均频谱曲线，代表该样本在各频率上的平均能量分布情况。LTAS 展示音频在时间和频率维度上的信噪比分布，可直观定位噪声集中的时间段或频段（如高频段突发噪声），辅助诊断具体失真类型，具有可视化整体音色平衡变化、直观表现音频的能量变化及截断现象等特点。

(2) LTAS 与的SNR<sub>p</sub>相关性

若某一音频样本在 LTAS 中表现出明显的频段能量削弱或突变现象，同时其感知信噪比得分较低，说明模型成功识别了对应的质量劣化特征，具有良好的感知一致性；

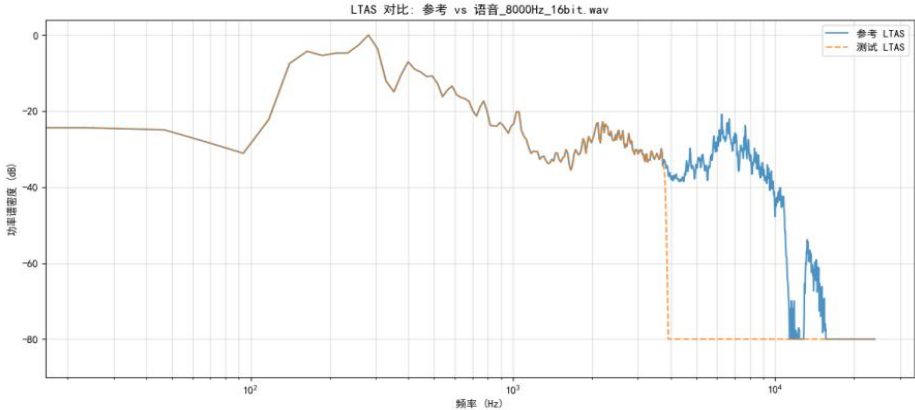
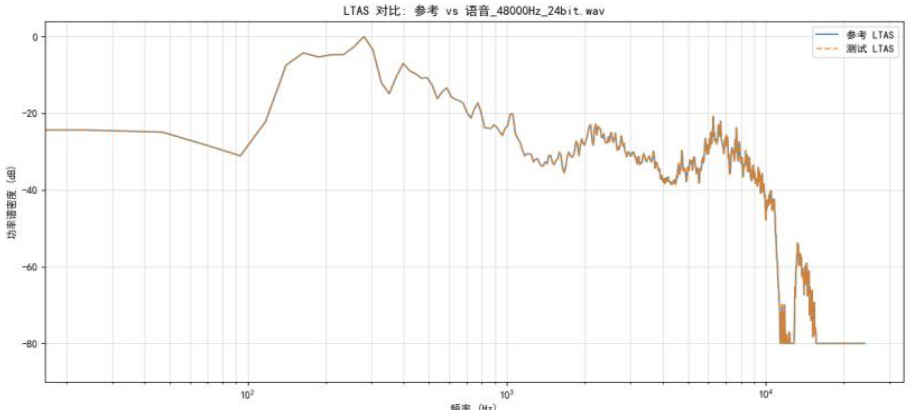
若 LTAS 出现能量异常而感知信噪比评分未显著下降，则提示模型对某些频段的失真识别能力不足，可能存在局部失真识别盲点；

反之，如果感知信噪比评分显著降低，但 LTAS 分布无明显异常，也可能表明模型对非频谱类失真（如瞬态、相位扰动等）的检测具有一定能力。

(3) 样本案例与对比分析

通过选取若干具有代表性的语音样本，比较其 LTAS 曲线与感知信噪比得分（SNR<sub>p</sub>），进一步验证模型在频谱分析与主观质量评估方面的一致性与准确性：

表 2.6 语音样本组 1 感知信噪比与 LATS 对比

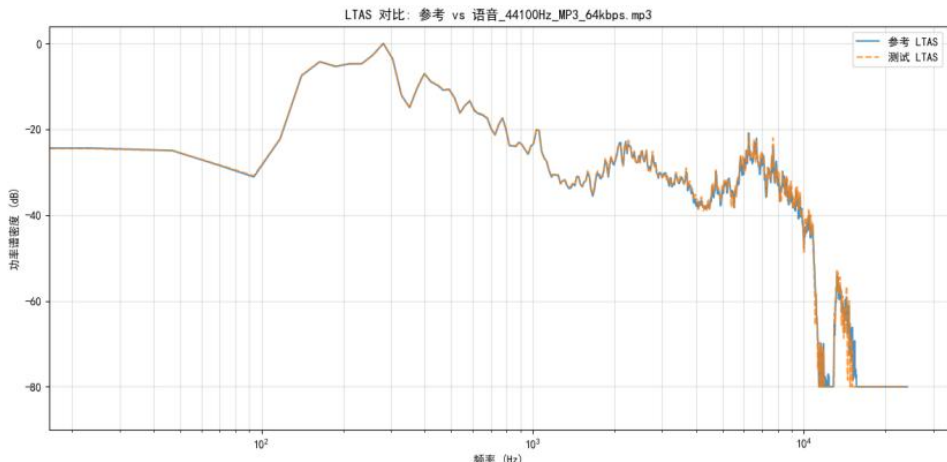
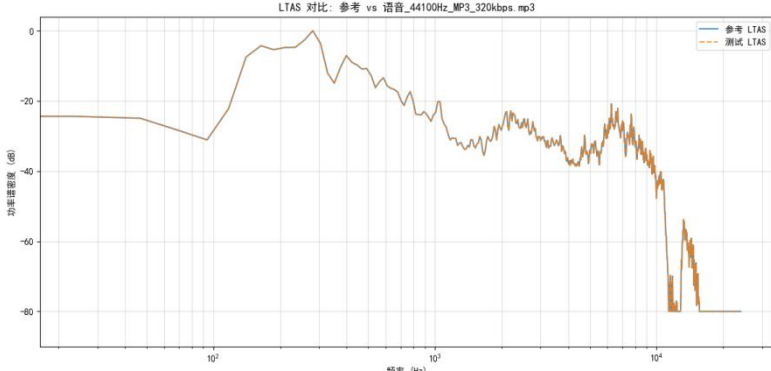
| 文件<br>名                               | SNR <sub>p</sub> | LATS                                                                                 |
|---------------------------------------|------------------|--------------------------------------------------------------------------------------|
| 语 音<br>_8000<br>Hz_16<br>bit.wa<br>v  | 13.4             |   |
| 语 音<br>_4800<br>0Hz_2<br>4bit.w<br>av | 146.3            |  |

样本组 1：语音\_8000Hz\_16bit.wav 与 语音\_48000Hz\_24bit.wav

语音\_8000Hz\_16bit.wav 的 LTAS 显示出明显的高频截断现象，频谱能量在 4 kHz 以上迅速衰减，失真较为严重；而 语音\_48000Hz\_24bit.wav 的 LTAS 曲线与参考原始音频在各频段高度吻合，几乎无能量损失。同时，后者的 SNRP 值远高于前者，表明模

型能准确反映实际频谱质量差异。

表 2.7 语音样本组 2 感知信噪比与 LATS 对比

| 文件<br>名                    | SNR <sub>p</sub> | LATS                                                                                |
|----------------------------|------------------|-------------------------------------------------------------------------------------|
| 语音_44100Hz_MP3_64kbps.mp3  | 20.8             |   |
| 语音_44100Hz_MP3_320kbps.mp3 | 66.6             |  |

**样本组 2：**语音\_44100Hz\_MP3\_64kbps.mp3 与 语音\_44100Hz\_MP3\_320kbps.mp3 在 MP3 编码样本中，64 kbps 版本在高频段的 LTAS 显著偏离原始音频，表现出一定程度的高频信息丢失，而 320 kbps 版本的 LTAS 与参考音频高度重合，音质保留较好。对应地，320kbps 样本的SNR<sub>p</sub>得分显著高于 64 kbps,进一步印证模型对压缩损失的感知能力。

通过对多个样本的SNR<sub>p</sub>与 LTAS 对比分析发现，LTAS 的吻合程度与SNR<sub>p</sub>大小呈正相关。当测试音频的 LTAS 曲线在各频段，尤其是高频段更为吻合时，意味着两者在频谱能量分布上更为相似，此时SNR<sub>p</sub>数值越高，表明音频质量受噪声干扰越小，越接近原始参考音频的质量。反之，若 LTAS 曲线偏离明显，则SNR<sub>p</sub>较低,音频质量受噪声影响较大。

这一实验结果充分说明，本模型能够有效捕捉音频在特定频段的能量损失与失真现象，具备较高的评估准确性与感知一致性。

## 七、问题三的模型建立与求解

### 7.1 多场景自适应音频编码优化方案建立

针对问题 3 所提出的需求，本文构建了一套多场景自适应编码优化系统。该系统结合音频内容识别模型、基于性价比的压缩效果预测回归模型，以及面向不同应用场景的权重策略，能够自动完成如下任务：

- (1) 识别原始音频类型——语音或音乐

(2) 提取关键音频特征并预测压缩后的性能指标

(3) 根据”移动播放”、”高保真收藏”与”通用下载”三种典型应用场景，智能推荐最优压缩参数组合——音频格式、采样率、比特率与比特深度

本文将该方案应用于附件 1 提供的原始音乐和原始语音样本，记录优化后的编码参数、输出文件大小与音质保真度，并与传统固定参数编码方案进行对比分析，从而验证该自适应方案在不同场景下的性能提升效果。

7.1.1 频谱特点和动态范围识别模块设计

为了实现对输入原始音频内容的高效感知与压缩决策支持，本文设计并实现了音频频谱与动态特征提取模块。该模块基于 Librosa 音频处理库，并结合 Scipy 的统计函数，能够从任意格式的音频输入中自动提取多维度频谱及统计特征，为后续音频类型判别与编码参数推荐提供基础特征支持。

(1) 模块输入与预处理

输入音频文件格式，系统在读取音频时保留其原始采样率，避免因重采样引入误差。系统默认采用单通道处理，并假定输入位深为 16bit，如有需要亦可扩展为动态读取。

(2) 频谱特征提取

本模块提取了以下与频谱相关的特征指标：

表 3.1 音频频谱相关的特征指标

| 使用场景  | 说明                                    |
|-------|---------------------------------------|
| 零交叉率  | 衡量音频信号过零点的频率，反映信号的频率结构，语音信号中常用于区分清浊音。 |
| 均方根能量 | 表示音频整体响度的能量分布。                        |
| 谱质心   | 衡量音频频谱重心位置，是描述音色明亮度的重要指标。             |
| 谱平坦度  | 表示频谱的平滑程度，可区分噪声与有调声音。                 |
| MFCC  | 提取 13 维梅尔频率倒谱系数，反映音频在感知域的频谱形状。        |
| 香农熵   | 对转换至 dB 域的频谱信息计算香农熵，用以评估频率分布的混乱程度。    |

(3) 统计特征与动态特性提取

为进一步提升模型感知能力，系统引入统计与动态类特征，包括以下特征：

表 3.2 音频频谱动态类特征指标

| 使用场景  | 说明                                                    |
|-------|-------------------------------------------------------|
| 偏度与峰度 | 刻画音频波形的非对称性与陡峭程度，有助于识别音频能量集中特性。                       |
| 动态范围  | 通过计算 RMS 的最大与最小值之差，反映音频从最弱到最强的响度变化幅度，特别适用于音乐类内容的质量评估。 |
| 音高    | 采用基于 STFT 的音高估计方法，取所有帧中有效音高的平均值，适用于旋律型信号分析。           |
| 音量    | 使用 RMS 能量的均值表示整体响度，与主观听感密切相关。                         |

(4) 模块输出

模块最终输出统一封装为特征字典形式，包含上述 12 项频谱与统计特征，作为后续音频分析的基础。

7.1.2 音频类型识别模块设计

在本节中，为实现对原始音频内容的自动化识别与分类，引入了 Google 开源的预训练模型 YAMNet。该模型基于 MobileNet v1 架构，已在 AudioSet 数据集上训练完成，能够对多达 521 类常见环境音事件进行多标签分类。YAMNet 具备高效、轻量的特点，适用于本项目对音频类型（语音、音乐、其他）的快速判断与分析。

实现了一个音频类型识别的任务，其核心目标是通过 YAMNet 模型来区分给定音频文件是否为语音或音乐。在此基础上，模型还会对音频中的主要成分进行分析，以确定

该音频文件是以语音为主，还是以音乐为主，或是包含混合成分。

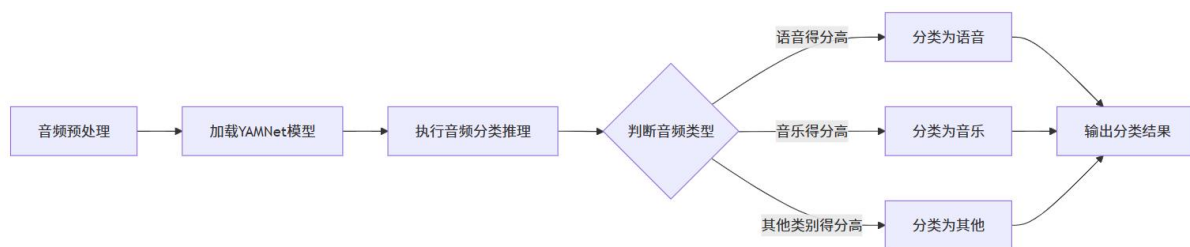


图 3.1 音频类型识别模块流程图

本系统整体流程包括四个核心步骤，依次为：音频预处理、模型推理与分类、结果判断、以及分数输出。该流程旨在从原始音频中准确提取语义特征，并实现音频类型（如语音、音乐等）的自动化识别与分类。具体流程如下：

（1）音频预处理：为保证输入音频与模型训练时的特征分布一致，采用 FFmpeg 工具对用户输入的任意格式音频进行标准化处理，包括：转换为 WAV 格式、统一采样率至 16 kHz、转为单声道，并确保输出位深为 16-bit PCM。此步骤旨在剥除非结构性差异，提高模型输入的一致性与鲁棒性。

（2）模型推理与分类：预处理后的音频数据输入至预训练的 YAMNet 模型中。YAMNet 构建于轻量级 MobileNet v1 网络之上，并已在大型环境声音数据集 AudioSet 上完成训练。模型内部采用 log-Mel 频谱图作为输入特征，通过多层卷积提取频率-时间域特征，从而输出多标签分类概率向量，覆盖 521 种音频事件标签。

（3）结果判断：针对模型输出的多维标签概率向量，系统依据得分高低判定音频主类型。若语音类向量概率最高，则音频被判定为语音类；若音乐类向量概率最高，则判定为音乐类；若其他类得分居前，则归类为环境音或未知类别。

（4）分数输出：为便于后续系统推荐模块使用与人工校验，模型的各主要标签的分类概率得分将作为附加输出记录。该分数向量不仅为最终分类结果提供概率依据，也可用于绘制音频特征热图或训练下游回归模型。

### 7.1.3 压缩参数的场景化推荐设计

本系统采用一种多阶段方法实现音频压缩参数的场景化推荐，其流程如图 xx 所示。

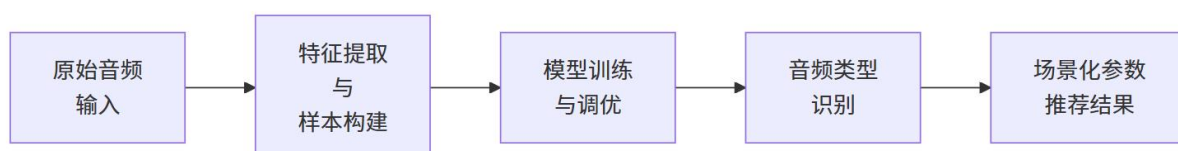


图 3.2 压缩参数的场景化推荐流程图

#### （1）数据预处理与特征工程

本研究基于附件 1 中提供的原始语音与音乐样本，构建了包含 177 条不同编码参数组合的音频数据集。针对原始音频及其压缩版本，提取了多维度的结构性特征用于建模与分析。

在特征工程阶段，引入了一个综合评价指标——场景性价比，其定义为信噪比 SNR 与文件大小比 Ref 的比值，即：

$$CPR_S = \frac{SNR}{Ref}$$

旨在衡量压缩音频在音质保真度与存储效率之间的整体表现。同时，剔除了与原始样本无关的数据记录，并对存在缺失值的条目进行了补全或删除处理，确保数据质量。

最终，模型选取的核心特征包括：压缩后文件大小比、信噪比以及计算得到的性价比指标。数据集按照 80%与 20%的比例划分为训练集与测试集，以保证模型训练与泛化



能力的评估。

考虑到不同编码格式在参数优化侧重点上的差异，本研究分别对各类编码格式构建了独立的子模型，以提高模型对不同压缩机制的适应性与预测精度。

## （2）场景化推荐策略

为实现面向不同应用场景的压缩参数推荐，本系统在回归模型设计中直接引入了场景权重信息，使得模型能够自动学习在不同使用场景下压缩性能的最优策略。用户选择场景后，系统据此将相应的权重配置（见第 5.1.4 节）作为特征输入之一，联合音频本身的结构性特征，用于模型预测。推荐流程如下：

首先，系统针对输入音频文件提取所需的结构性特征，并识别其音频格式类型。随后，根据用户指定的应用场景，自动加载相应的场景权重向量，作为模型输入的一部分。训练阶段中，模型已通过大量不同场景与参数组合的数据进行学习，因此能够根据当前输入音频及场景偏好，预测出性能最优的参数组合。

表 3.3 不同场景下的加权策略

| 使用场景    | 关注点     | 加权策略说明                  |
|---------|---------|-------------------------|
| 移动流媒体播放 | 文件体积小   | 优先推荐压缩率高的编码方案           |
| 高保真音乐收藏 | 音质保真度最优 | 优先推荐 SNR 较高、动态范围大的参数组合  |
| 通用下载与存储 | 综合性价比   | 平衡 SNR 与文件大小，按性价比排序推荐结果 |

与传统的加权打分方法相比，该策略无需显式构建评价函数或归一化过程，而是通过机器学习模型内部完成场景权重与特征之间的非线性建模，使推荐结果更具适应性与泛化能力。

## （3）模型选择与训练

本模型面向音频压缩参数推荐任务，需对采样率、比特率与比特深度等连续型目标变量进行回归预测。考虑到音频特征与最优编码参数之间可能存在的非线性映射关系，以及输入特征维度较高的情况，最终选择采用随机森林回归模型进行建模。

随机森林是一种集成学习算法，具有较强的非线性拟合能力和良好的泛化性能，适用于复杂高维的数据建模任务。其核心优势包括：

（i）建模能力强：可有效刻画特征与目标变量间的非线性关系，适配音频特征与压缩参数之间的复杂映射；

（ii）特征自动选择：模型可评估各输入特征对预测目标的重要性，有助于识别关键音频特征，如信噪比、文件大小等；

（iii）鲁棒性高：对缺失值、异常值不敏感，能适应实际音频数据中可能存在的质量问题；

（iv）易于训练与调参：相较神经网络类模型，训练速度更快，尤其适合中小规模的结构化特征数据集。

为提升压缩参数性价比预测模型的准确性与泛化能力，本文基于训练数据采用随机森林回归模型分别构建了采样率预测模型、比特深度预测模型、以及比特率预测模型。考虑到不同场景对音频编码优化目标存在差异，本研究在移动流媒体、高保真音乐收藏、通用下载/存储三类典型场景下分别独立训练模型。

在模型训练阶段，为进一步提升预测性能，我们针对每一类目标参数以及三种典型使用场景分别训练子模型，并使用网格搜索对模型超参数进行系统调优。调参的关键目标是最小化负均方误差，从而获得更优的泛化能力。具体搜索的超参数如下表：

表 3.4 模型超参数范围说明

| 超参数名称        | 搜索范围                | 说明              |
|--------------|---------------------|-----------------|
| n_estimators | [50, 100, 200, 300] | 决策树数量，平衡性能与时间消耗 |
| max_depth    | [5, 10, 20, None]   | 树最大深度，防止过拟合     |



|                   |               |               |
|-------------------|---------------|---------------|
| min_samples_split | [2, 4, 6, 10] | 内部节点划分所需最小样本数 |
| min_samples_leaf  | [1, 2, 4, 6]  | 叶子节点最小样本数     |
| cv（交叉验证折数）        | 2             | 简化计算时间，保持鲁棒性  |

通过上述模型选择与训练策略，系统在保持泛化性能的同时，实现了对多场景下压缩参数的有效预测与推荐。

## 7.2 多场景自适应音频编码优化方案求解

### 7.2.1 模型数据集构建

在本研究中，为系统性评估不同编码配置对音频压缩效果的影响，针对原始音频数据设计并实施了多组压缩参数组合的转码处理实验。所选参数涵盖音频编码的主要控制项，具体包括：

表 3.5 数据集参数设置

| 参数   | 参数设置                                                  |
|------|-------------------------------------------------------|
| 采样率  | 8000Hz、16000 Hz、22050 Hz、32000 Hz、44100 Hz 与、48000 Hz |
| 比特深度 | 8bit、16bit、24bit                                      |
| 比特率  | 64kbps、96kbps、128kbps、192kbps、256 kbps、320 kbps       |
| 编码格式 | MP3、AAC、WAV                                           |

在上述参数空间内，对原始音频文件进行批量转码，生成对应的压缩文件。为构建预测模型，从压缩音频中提取模型所需的参数特征，基于与原始音频的对比计算实际信噪比，以量化音质保真度。最终，结合压缩比与信噪比，计算“性价比”指标，为后续建模与参数推荐系统提供标准化训练数据支持。

### 7.2.2 超参数调优

基于 6.1.2 节中设置的超参数范围，对每组参数组合均通过 5 折交叉验证评估预测性能，并记录在不同场景下的最优配置。最终，三类场景下各参数项的最优模型超参数如下表所示：

表 3.6 最优超参数配置

| 参数项         | 最优参数配置                                                                                  |
|-------------|-----------------------------------------------------------------------------------------|
| 采样率         | {'n_estimators': 100, 'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2}    |
| WAV 比特深度    | {'n_estimators': 50, 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 3}  |
| MP3/AAC 比特率 | {'n_estimators': 100, 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2} |

### 7.2.3 多场景最佳参数优化方案求解

为验证所提出的多场景音频编码优化方案的实际效果，本文选取附件 1 中的两类原始音频文件作为测试样本，分别为“原始语音\_48kHz\_24bit.wav”与“原始音乐\_48kHz\_24bit.wav”。基于前述构建的参数推荐模型与场景加权策略，分别在移动流媒体、高保真音乐收藏、通用下载/存储三类典型应用场景下对原始音频进行编码参数推荐与压缩处理。得到结果如下表：

#### （1）原始语音的最佳参数推荐

表 3.7 原始语音的最佳参数推荐

| 使用场景  | 格式  | 采样率<br>(Hz) | 比特深度<br>/ 比特率 | 预测文件<br>大小 (KB) | 预测信噪比<br>(dB) | 场景性价比得分 |
|-------|-----|-------------|---------------|-----------------|---------------|---------|
| 移动流媒体 | WAV | 22050       | 8-bit         | 128.58          | 36.25         | 236.59  |
|       | MP3 | 22050       | 65 kbps       | 47.39           | 24.1          | 426.71  |

|         |     |       |          |        |        |        |
|---------|-----|-------|----------|--------|--------|--------|
| 高保真音乐收藏 | AAC | 8000  | 16 kbps  | 12.46  | 30.35  | 2043.9 |
|         | WAV | 48000 | 24-bit   | 839.29 | 146.38 | 146.37 |
|         | MP3 | 48000 | 323 kbps | 235.36 | 69.94  | 249.4  |
| 通用下载/存储 | AAC | 48000 | 79 kbps  | 58.46  | 35.35  | 507.47 |
|         | WAV | 22050 | 8-bit    | 128.58 | 36.25  | 236.59 |
|         | MP3 | 22050 | 65 kbps  | 47.39  | 24.1   | 426.71 |
|         | AAC | 8000  | 16 kbps  | 12.46  | 30.35  | 2043.9 |

移动流媒体场景下，AAC 格式在 8000 Hz 采样率与 16 kbps 比特率配置下表现最佳，其预测文件大小仅为 12.46 KB，信噪比达 30.35 dB，综合性价比高达 2043.9，显著优于 MP3 和 WAV。同等体积下 AAC 提供了更高的音质，是资源受限场景下的首选。

高保真音乐收藏场景中，尽管 WAV 格式(48kHz / 24-bit)带来最高的信噪比(146.38 dB)，但因其文件体积极大(839.29 KB)，导致性价比评分仅为 146.37。相较而言，AAC(48kHz / 79 kbps)在提供较优音质(35.35 dB)基础上，控制文件大小为 58.46 KB，性价比达 507.47，在质量和存储之间达成良好平衡，优于 MP3 与 WAV。

通用下载/存储场景中，与移动流媒体配置完全一致，AAC 格式同样占优，表明该组合具有良好的通用适应性和迁移性。

## (2) 原始音乐的最佳参数推荐

表 3.8 原始音乐的最佳参数推荐

| 使用场景    | 格式  | 采样率<br>(Hz) | 比特深度<br>/ 比特率 | 预测文件<br>大小 (KB) | 预测信噪比<br>(dB) | 场景性价<br>比得分 |
|---------|-----|-------------|---------------|-----------------|---------------|-------------|
| 移动流媒体   | WAV | 8000        | 8-bit         | 78.2            | 26.25         | 472.13      |
|         | MP3 | 44100       | 64 kbps       | 78.61           | 24.29         | 434.59      |
|         | AAC | 8000        | 32 kbps       | 40.74           | 24.70         | 852.81      |
| 高保真音乐收藏 | WAV | 48000       | 24-bit        | 1406.29         | 146.26        | 146.26      |
|         | MP3 | 48000       | 257 kbps      | 314.29          | 33.92         | 151.79      |
|         | AAC | 48000       | 140 kbps      | 172.14          | 35.73         | 291.93      |
| 通用下载/存储 | WAV | 8000        | 8-bit         | 78.2            | 26.25         | 472.13      |
|         | MP3 | 44100       | 64 kbps       | 78.61           | 24.29         | 434.59      |
|         | AAC | 8000        | 32 kbps       | 40.74           | 24.70         | 852.81      |

在移动流媒体场景中，AAC(8000 Hz / 32 kbps)以中等的比特率实现了 24.71 dB 的信噪比，文件大小仅 40.74 KB，性价比达 852.82，远高于 MP3 和 WAV，凸显了 AAC 在低码率条件下处理音乐内容的编码效率。

对于高保真音乐收藏，WAV 格式虽以最高音质(146.27 dB)占据绝对优势，但文件大小也最大(1406.29 KB)。AAC(48kHz / 140 kbps)在音质(35.74 dB)和文件大小(172.14 KB)之间取得了良好折中，性价比评分为 291.94，相比 MP3 更具存储效率，适用于对高音质有要求但又希望控制存储成本的用户。

通用下载/存储下，结果与移动流媒体一致，AAC 的优势持续显现，说明其具有广泛适配性。

总体而言，AAC 格式在各类场景中均表现出较高的压缩效率和良好的音质保真性，特别适合移动端与通用使用需求。而在对音质极端敏感的高保真收藏场景中，WAV 格式虽然性价比不高，但在保真度上的绝对优势依旧不可替代。MP3 格式在大多数场景中处于中间地位，表现稳定但缺乏显著优势。

本系统的多场景自适应音频编码优化方案能够根据使用场景智能推荐最优参数组合，有效兼顾存储效率与音频质量，具备良好的应用前景。

7.2.3 多场景最佳参数优化方案与固定参数方案对比

我们根据本节推荐的最佳参数方案生成了该方案参数下的音频，并将其放入上节的评价体系下进行评价，进一步验证了优化方案的有效性。

首先，固定参数方案通常依赖于预设的参数配置，不会根据具体的音频类型或场景需求进行动态调整。为了进一步验证我们的优化方案的效果，我们根据本节推荐的最佳参数方案生成了相应的音频文件，并将这些文件放入上节的评价体系中进行评估。以下选取上节最优的两个方案与本节方案对比，结果如下：

表 3.9 语音、音乐方案评分对比

| 文件名                           | 总分       | 文件名                      | 总分       |
|-------------------------------|----------|--------------------------|----------|
| output_MP3_22050Hz_65kbps.mp3 | 28.25892 | 音乐                       |          |
| 语音_44100Hz_AAC_96kbps.aac     | 23.2607  | _44100Hz_MP3_64kbps.mp3  | 11.82056 |
| 语音_44100Hz_AAC_128kbps.aac    | 18.39878 | 8000_32.m4a              | 7.160745 |
|                               |          | 音乐                       |          |
|                               |          | _44100Hz_MP3_128kbps.mp3 | 6.42768  |

总体来看，本节的优化方案在上节评价体系下也有不俗的表现，并且能够提供更符合实际需求的音频编码方案，尤其是在文件大小和压缩效率要求较高的应用场景中，如移动流媒体和语音识别等。虽然在某些情况下音质可能略有妥协，但其适应性和压缩性能的优化为实际应用提供了更高的价值。

7.3 多场景自适应音频编码优化方案评价

7.3.1 模型指标评估

在语音音频数据集上，分别构建了采样率、比特深度与比特率的回归预测模型，其性能评估指标如下表所示。

表 3.10 不同编码参数回归模型在语音音频数据集上的性能评估结果

| 模型名称       | 决定系数 $R^2$ | 平均绝对误差 MAE | 均方根误差 RMSE |
|------------|------------|------------|------------|
| 采样率模型      | 0.6218     | 7233.5833  | 10425.7387 |
| WAV 比特深度模型 | 0.5380     | 6.1733     | 6.7822     |
| MP3 比特率模型  | 0.9777     | 5.4936     | 9.0103     |
| AAC 比特率模型  | 0.9817     | 3.8265     | 5.6963     |

从结果来看，AAC 与 MP3 比特率模型均取得了较高的拟合效果（ $R^2$  分别为 0.9817 和 0.9777），说明比特率是影响语音压缩性价比最关键的因素之一。比特率的调整直接影响编码后的音质清晰度与文件大小，尤其在语音场景中，过低比特率可能导致语义丢失，而过高则带来存储冗余，故其影响最为显著。

相比之下，采样率模型的表现一般（ $R^2 = 0.6218$ ），这可能与语音信号本身对频带信息的依赖较弱有关。在常见的 16–24kHz 范围内，语音内容的还原已基本充分，进一步提升采样率对最终音质和性价比影响有限。

WAV 比特深度模型的表现最差（ $R^2 = 0.5380$ ），说明该特征在语音场景下与目标性价比之间的相关性较弱。原因可能在于语音信号的动态范围本身较窄，提升比特深度并不会带来显著音质提升，反而增加了编码后的文件大小，导致模型难以准确学习该特征的有效性。

在音乐音频数据集中，模型性能评估结果如下表所示：

表 3.11 不同编码参数回归模型在音乐音频数据集上的性能评估结果

| 模型名称       | 决定系数 $R^2$ | 平均绝对误差 MAE | 均方根误差 RMSE |
|------------|------------|------------|------------|
| 采样率模型      | 0.7619     | 3258.3056  | 7481.5377  |
| WAV 比特深度模型 | 0.5844     | 4.6800     | 4.9379     |
| MP3 比特率模型  | 0.9977     | 1.6031     | 1.9257     |

|           |        |        |        |
|-----------|--------|--------|--------|
| AAC 比特率模型 | 0.9896 | 3.2963 | 4.3609 |
|-----------|--------|--------|--------|

音乐类音频中，比特率依然是决定性因素。MP3 比特率模型取得了最优表现 ( $R^2 = 0.9977$ )，AAC 模型同样表现稳定 ( $R^2 = 0.9896$ )，表明比特率对音乐压缩后主观音质与文件大小的平衡至关重要。音乐信号频率成分丰富、动态变化大，因而对压缩算法的控制尤为敏感。

采样率模型的表现优于语音场景 ( $R^2 = 0.7619$ )，说明在音乐场景下，高采样率更能有效保留高频细节，进而提升整体音质，因此模型具备更强的拟合能力。

WAV 比特深度模型依旧表现较差， $R^2$  仅为 0.5844，尽管略优于语音数据，但整体预测能力仍然不足。这表明比特深度虽能在一定程度上提升音乐保真度，但其对压缩后性价比的贡献较为有限，尤其在压缩场景下，其影响被编码算法所掩盖。

综合语音与音乐两个数据集的建模结果可以得出，比特率是影响压缩性价比的关键参数，尤其是在 MP3 和 AAC 格式中，模型对比特率的拟合能力极强。采样率在音乐场景中扮演着更重要的角色，而语音内容对此参数的敏感度相对较低。相比之下，比特深度对整体压缩性价比的贡献最为有限，无论是处理语音还是音乐，模型都难以从中学习到有效的特征。因此，在后续的音频编码推荐策略中，应着重考虑音频类型，并以此为依据，优先调整不同的关键参数维度以实现优化。

### 7.3.2 方案评价和改进建议

该方案能够应对实际中面对的各种音频需求，特别是在高保真音质与文件大小之间找到平衡，适合不同场景的使用需求。但仍有以下方面需要改进：

#### (1) 数据集扩展

当前的数据集较为有限，应该尽可能扩展数据集，音频类型和压缩参数的组合上，模型的训练数据量不足以覆盖所有可能的情况，以提升模型在实际应用中的表现。

#### (2) 模型优化

尽管随机森林回归在许多场景中表现良好，但它在面对复杂的非线性关系时可能无法完全捕捉到最佳模式。考虑引入深度学习方法，以处理更复杂的音频特征，进一步提升音频的预测性能。

## 八、问题四的模型建立与求解

### 8.1 噪声特征提取与选择

为了实现自适应去噪，首先需要对音频信号中的噪声进行分类，为每种噪声选取特征参数，以便进行准确的识别和量化。本模型采用短时傅里叶变化的时频分析法，并从中提取一系列声学特征来区分不同噪声类型。

#### 8.1.1 噪声类型特征分析

本研究主要关注在实际音频中常见的三类噪声：背景噪声、突发噪声和带状噪声。它们的典型特征如下：

表 4.1 各类型噪声特征

| 噪声类型 | 特征描述          | 可视化特征       |
|------|---------------|-------------|
| 背景噪声 | 能量平稳、频率分布广    | 全频段低强度噪声    |
| 突发噪声 | 时间局部性强、能量集中   | 某时间窗口出现高能量峰 |
| 带状噪声 | 特定频段持续存在、频率稳定 | 某一频段强烈能量带   |

#### 8.1.2 噪声特征参数选取

根据上述噪声类型的典型特征，选取了不同的声学特征去识别它们。每种噪声类型对应的声学特征，以及特征代表的含义如下表所示：

表 4.2 各噪声参数介绍及公式

| 噪声类型 | 量化参数          | 计算公式                                                                            | 说明                                                               |
|------|---------------|---------------------------------------------------------------------------------|------------------------------------------------------------------|
| 背景噪声 | 对数能量方差<br>LEV | $Energy_t = \sum_f  S(f, t) ^2$                                                 | 数值相对较低, 表示能量稳定                                                   |
|      | 最大帧峰度<br>KM   | $Kurtosis = \frac{1}{N} \sum_{i=1}^N \left( \frac{x_i - \mu}{\sigma} \right)^4$ | 其中 $\mu$ 是均值, $\sigma$ 是标准差。高于 3 则为尖峰型分布。数值显著地高于 3, 表示存在尖锐脉冲。    |
| 突发噪声 | 能量峰值率 EP      | $E_t = \sum_{n=1}^N y_t[n]^2$                                                   | 用 <code>find_peaks</code> 检测峰值, 设置相对于中位能量的显著性门限。数值相对较高, 表示瞬态事件频繁 |
|      | 平均谱平坦度<br>SF  | $SFM = \frac{(\prod_{i=1}^N x_i)^{1/N}}{\frac{1}{N} \sum_{i=1}^N x_i}$          | 其中 $x_i$ 是频谱分量的幅度, 几何均值除以算术均值。数值非常接近 0, 表示能量高度集中                 |
| 带状噪声 | 平均谱熵 SEM      | $Entropy = - \sum_{i=1}^N p_i \log_2(p_i)$                                      | $p_i$ 为归一化功率<br>数值相对较低, 表示频谱结构简单                                 |
|      | 平均谱峰数量<br>NSP | $NSP = \frac{1}{N} \sum_{i=1}^N n_i$                                            | 共有 N 个频谱, 第 i 个频谱中谱峰数量为 $n_i$<br>数值相对较高, 表示存在窄带能量峰               |
|      | 最大谱峰显著度 MPP   | $MPP = \max\{p_1, p_2, \dots, p_M\}$                                            | 存在非常高的数值, 表示有突出能量峰                                               |
|      | 谐波存在性 HD      | $HD = \sum_{k=2}^K \frac{A_k}{A_0}$                                             | K 为考虑的最高谐波次数<br>检测到谐波 (作为辅助判断)                                   |

通过关注提取的声学特征的数值, 结合决策表, 便可以达到识别和分类噪声类型的目的。

## 8.2 噪声识别模型的建立

为实现对样本音频中不同类型噪声成分的识别, 我们建立了一个基于规则的噪声识别模型。该模型通过分析音频信号中提取的多种特征参数, 与设定的阈值进行逻辑比较, 从而判断噪声的种类。

### 8.2.1 输入向量 X

模型的输入为特征参数向量:

$$X = [x_1, x_2, \dots, x_n]$$

其中, 每个分量 $x_i$ 代表第*i*个噪声特征参数, 例如 $x_1 = LEV$ ,  $x_2 = KM$ 等。

### 8.2.2 模型参数 V

模型的核心参数是设定的阈值, 这些阈值定义了决策边界。由于样本数量较少, 这些阈值是根据先验经验来确定的。

$$V = [t_1, t_2, t_3, \dots, t_m]$$

### 8.2.3 决策规则

模型核心的决策规则定义为：

$$\text{If } x_i > V_i \text{ or } x_j < V_j \Rightarrow \text{检测到某类噪声}$$

具体的规则设定如下：

$$\text{若 } x_2 > 15.0 \text{ 或 } x_3 > 3.0 \text{ 则存在突发噪声} \quad (1)$$

$$\text{若 } x_4 < 0.01 \text{ 或 } x_5 < 4.5 \text{ 或 } x_6 \geq 50 \text{ 或 } x_8 = \text{True} \text{ 则存在带状噪声} \quad (2)$$

$$\text{若 } x_1 < 3.0 \text{ 则可能存在背景噪声} \quad (3)$$

### 8.2.4 输出结果

定义模型最终输出向量  $\mathbf{W}$ ，用于指示当前音频中检测到的噪声类型。

$$\mathbf{W} = \{Y_i, Y_j, Y_k\}$$

$$Y_i = \begin{cases} 1, & \text{存在背景噪声} \\ 0, & \text{不存在背景噪声} \end{cases}$$

$Y_j, Y_k$ 同理，用于指示突发噪声与带状噪声。

## 8.3 自适应去噪模型建立

为实现对混合噪声音频信号的高效降噪处理，建立一个基于噪声识别驱动的自适应去噪系统模型。模型的核心思想是构建一个模块化的处理流程，根据识别出的噪声成分激活相应的处理模块，并自适应地调整处理参数。模型主要包括四个功能模块：噪声识别模块、降噪策略模块、自适应参数调节模块，实现针对不同噪声类型自动选择最佳去噪方法。

### 8.3.1 噪声识别模块

算法的起点是噪声识别模块的输出结果噪声类型向量。使用 8.2 中的噪声识别模型，对音频特征参数进行逻辑判断，输出噪声类型向量  $\mathbf{W}$ ，自动识别是否存在突发噪声、带状噪声和背景噪声，并驱动后续降噪路径。

### 8.3.2 降噪策略模块

根据识别出的噪声类型，依次调用对应的算法进行处理。通常，处理顺序对结果影响很大，由此使用级联处理的方式，根据噪声类型待处理的优先级推荐处理顺序，算法按待顺序调用。

优先处理突发噪声，因其高度非平稳，会干扰后续噪声估计与处理，抑制它能为后续步骤提供更纯净信号。接着处理带状噪声，其频率集中，在突发噪声去除后更易检测滤除，也利于准确估计背景噪声。最后处理背景噪声，主要结构性噪声（突发、带状）被抑制后，针对平稳噪声的处理方法对残留宽带背景噪声更有效。

建立自适应去噪系统模型，流程图如下：

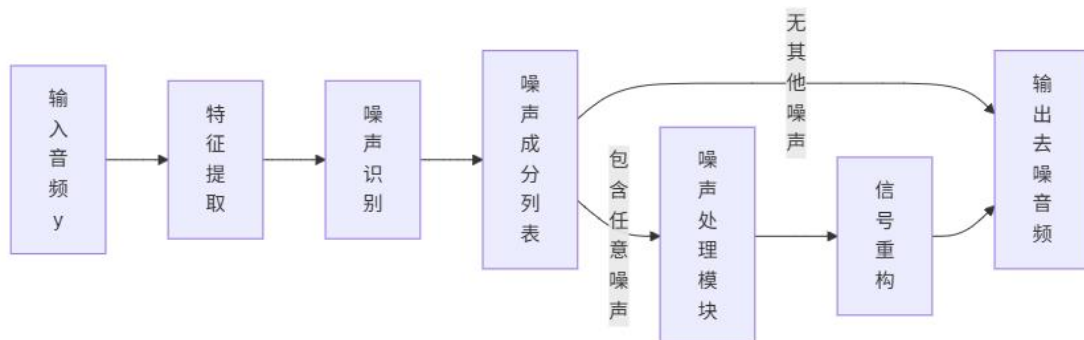




图 4.1 自适应去噪系统模型流程图

下图为噪声处理模块的内部流程：

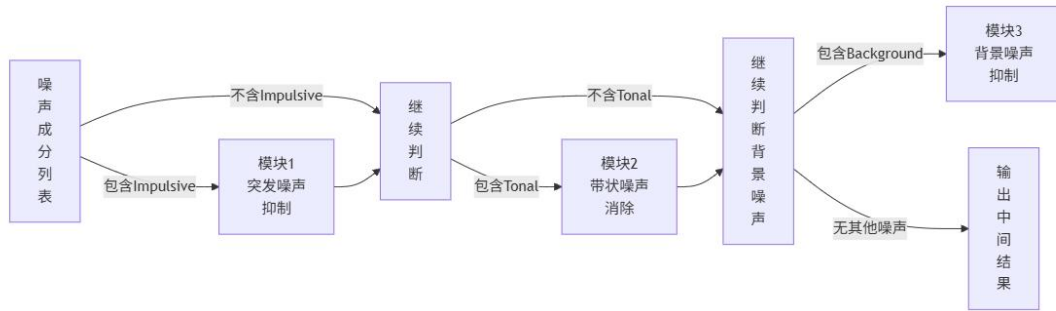


图 4.2 噪声处理模块的内部流程图

各模块仅在对应噪声类型被识别时激活，实现模块化执行与流程灵活调整。

### 8.3.3 自适应参数调节模块

为提升处理效果，引入参数优化模块，采用网格搜索方法，在一个预先定义的、由各参数离散候选值构成的多维参数空间中，寻找各种可能的参数组合。对于每个组合，使用降噪策略模块处理音频，并通过估计信噪比作为目标函数来评估处理效果。

优化目标为使估计的信噪比（SNR）最大化，SNR 的求解函数在问题二建模中已经提出，此处不再赘述。

## 8.4 噪声识别模型求解

### 8.4.1 时频分析及噪声特征参数

选用短时傅里叶变换作为时频分析工具。短时傅里叶变换将信号分成短时帧，对每一帧进行傅里叶变换，得到信号的频谱随时间变化的表示，即语谱图。颜色代表信号强度，颜色越暖表示强度越高，反之。

对频谱图进行分析：

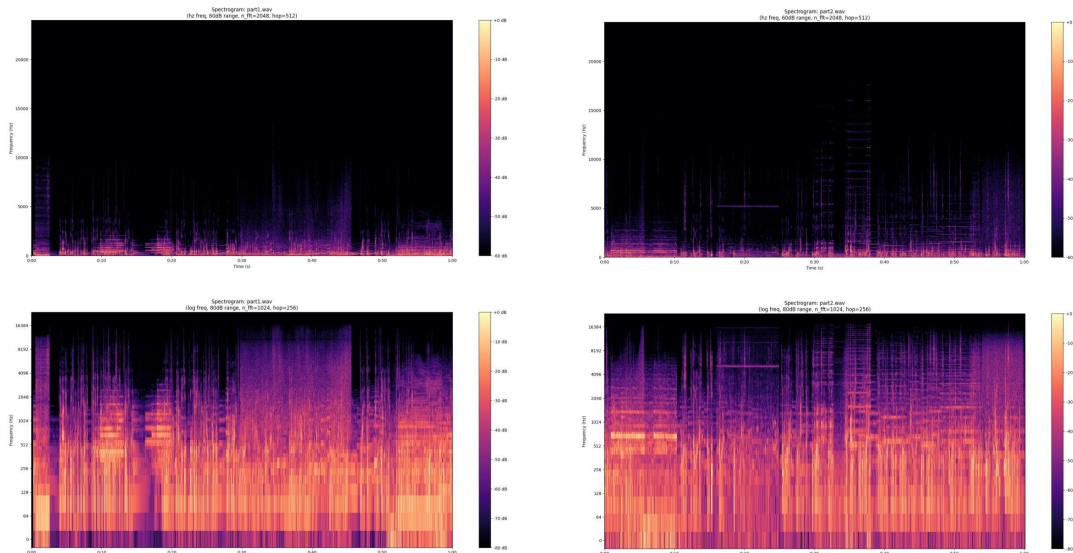


图 4.3 图 4.4 part1\_wav 组和 part2\_wav 组频谱图

两张图中都可以看到一些垂直的亮线或短暂的能量爆发。它们表现为持续时间短、跨越一定频率范围的事件。图中显示出一条水平亮线，中心频率在 5kHz 附近，这个特征极其显著，且在这段时间内持续存在。推测两段音频中可能含有突发噪声和带状噪声。

### 8.4.2 噪声识别模型求解

根据 8.1.2 中的公式，计算音频样本的噪声特征参数，参数如表：

表 4.3 音频样本的噪声特征参数

|             | part1.wav   | part2.wav   |
|-------------|-------------|-------------|
| 对数能量方差 LEV  | 4.018231    | 2.9650848   |
| 最大帧峰度 KM    | 24.20341    | 20.30361    |
| 能量峰值率 EP    | 3.681369936 | 3.942290632 |
| 平均谱平坦度 SF   | 0.002217294 | 0.000615506 |
| 平均谱熵 SEM    | 3.8628657   | 4.206021    |
| 平均谱峰数量 NSP  | 100         | 98          |
| 最大谱峰显著度 MPP | 1765.915    | 1311.185    |

将得到的噪声特征参数作为输入向量，输入噪声识别模型得到结果：

表 4.4 噪声识别模型结果

|      | part1.wav | part2.wav |
|------|-----------|-----------|
| 噪声类型 | 突发噪声、带状噪声 | 突发噪声、带状噪声 |

8.4.3 自适应去噪模型求解及结论

根据 8.4.2 中识别的噪声类型，为音频选择合适的去噪方案，并通过调整去噪参数，以达到最优的去噪参数组合。（具体参数见附件）

表 4.5 去噪前后信噪比对比

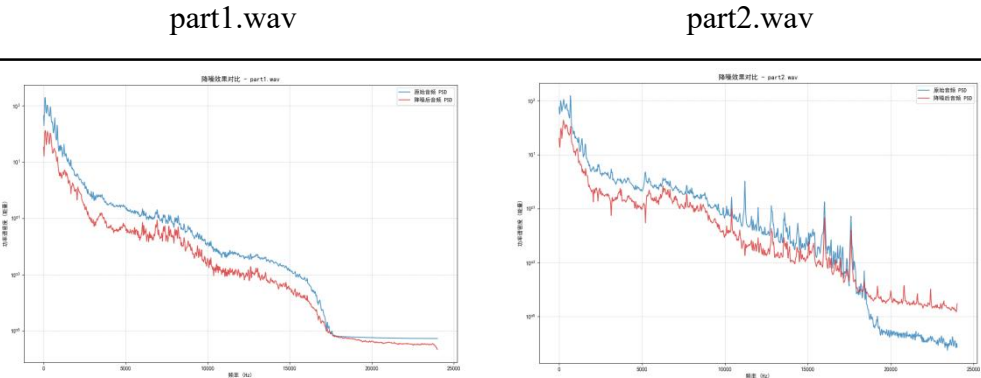
|         | part1.wav | part2.wav |
|---------|-----------|-----------|
| 原始音频信噪比 | 15.06 dB  | 14.67 dB  |
| 去噪后信噪比  | 18.55 dB  | 20.77 dB  |
| 提升信噪比   | +3.49 dB  | +6.1 dB   |

功率谱密度可以清晰地展示音频中噪声的频率分布情况，不同类型的噪声具有不同的功率谱密度特征。在进行音频降噪处理后，通过比较处理前后音频信号的功率谱密度，可以直观地评估降噪效果。如果降噪处理有效，那么噪声所在频率段的功率谱密度应该明显降低，而有用音频信号的功率谱密度则应尽量保持不变。

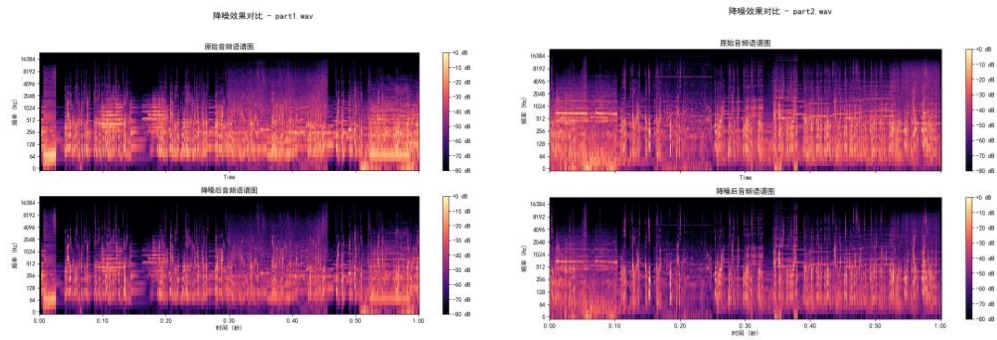
波形图能够直接呈现音频信号的幅度随时间的变化情况。通过观察波形的振幅大小，可以了解音频信号的强弱。在噪声处理中，这有助于判断噪声对音频信号整体幅度的影响。例如，如果噪声是突发的强脉冲，在波形图上会表现为明显的高幅值尖峰，与正常音频信号的幅度形成鲜明对比，从而帮助确定噪声的出现位置和强度。

通过对比图实现音频经过降噪模型处理前后降噪幅度的可视化。

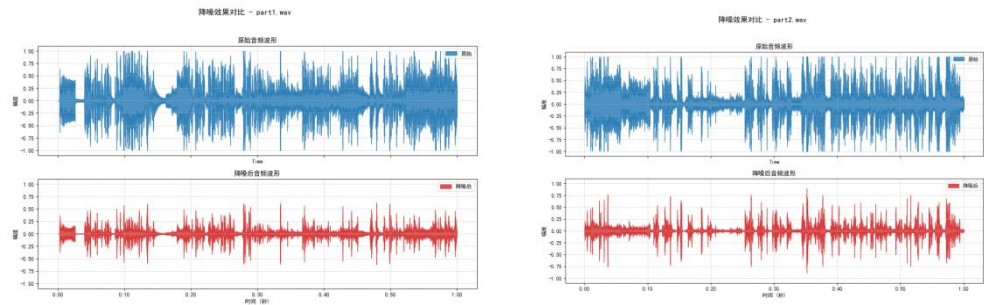
功率谱密度对比图



语谱图对比图



波形对比图



结论

结合三组对比，降噪模型有效地降低了带状噪声的整体能量水平，并有效抑制了突发噪声，同时较好地保留了主要信号结构。

结合三组对比，降噪模型准确地识别并消除在特定时间段出现的带状噪音，大幅降低了突发噪音的幅度，同时主要信号结构得以保留。

去噪后的新的 wav 文件见附件。

8.5 模型评价

8.5.1 模型优势

(1) 模型的核心优势在于其模块化设计，能针对性地处理不同噪声。当音频包含以下几种清晰噪声的组合时，能取得优秀的效果：相对平稳的背景噪声、清晰的突发噪声与强度足够、频率稳定、频带很窄的带状噪声。

(2) 噪声符合模块假设，当实际噪声与各处理模块的假设非常吻合时，模型效果最好：突发噪声在小波域是瞬态和稀疏的、带状噪声频率稳定且带宽很窄、背景噪声在某个时间段内统计特性是平稳或缓慢变化的。

(3) 当噪声处于中等噪声水平时，固定阈值规则检测系统更有可能正确识别存在的噪声类型，各个去噪模块可以使用通过网格搜索找到突出的最优参数，从而降低引入明显信号失真和处理痕迹的风险。

8.5.2 模型局限性

(1) 噪声检测阶段：

固定阈值：噪声识别模型中的阈值存在一定特殊性，它们可能只对特定音频有效。

不同的噪声强度：微弱的突发噪声可能低于峰度阈值，同时高强度的背景噪声可能错误地触发突发噪声或带状噪声的检测。

噪声与信号在频谱上重叠：具有类似特性的音频很可能会在抑制噪声的同时也抑制目标语音。

(2) 自适应去噪系统模块：

顺序处理与缺乏反馈：固定的处理顺序在面对其他音频样本时不一定是最佳的。

去噪处理重叠性：较前模块的处理痕迹可能会对后续阶段产生负面影响。

## 参考文献

- [1] 李军, 杨震. 基于客观评价模型的音频编码器性能分析与优化[J]. 电视技术, 2010, 34(S1): 5-8.
- [2] 杜军, 张雪冬. 基于改进经验模态分解和谱减法的自适应语音增强算法[J]. 信号处理, 2015, 31(10): 1234-1240.
- [3] 王超, 吴镇扬, 贾珈. 基于卷积神经网络的音频场景分类方法研究[J]. 计算机应用研究, 2018, 35(10): 3157-3161.
- [4] A. B. Spanias, "Speech coding: A tutorial review," Proceedings of the IEEE, vol. 82, no. 10, pp. 1541-1582, Oct. 1994.
- [5] M. Bosi and R. E. Goldberg, Introduction to Digital Audio Coding and Standards. Norwell, MA, USA: Kluwer Academic Publishers, 2003.
- [6] S. S. Shylaja, P. K. S. Kumar, and P. N. Shankar, "Audio scene classification using YAMNet and spectrogram analysis," in 2021 International Conference on Advances in Computing, Communication, and Control (ICAC3), Mumbai, India, 2021, pp. 1-6.
- [7] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
- [8] J. Herre and J. D. Johnston, "Enhancing the performance of perceptual audio coders by using temporal noise shaping (TNS)," in AES 101st Convention, Los Angeles, CA, USA, Nov. 1996, Paper 4360.

## 附录

```
#####问题三#####
#####3.1 音频类型识别模型.py
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import librosa
import pandas as pd
import os
import subprocess
import argparse
import tempfile

# ----- 使用 ffmpeg 转换为兼容 WAV -----
def convert_to_wav(input_path, output_path, target_sr=16000):
    """
    使用 FFmpeg 将任意音频格式转为 target_sr (默认 16kHz) mono WAV。
    返回 True 表示成功, False 表示失败。
    """
    print(f"正在使用 FFmpeg 将 '{os.path.basename(input_path)}' 转换为 16kHz Mono WAV...")
    cmd = [
        'ffmpeg', '-y',          # 覆盖输出文件而不询问
        '-i', input_path,        # 输入文件
        '-ac', '1',              # 设置声道数为 1 (mono)
        '-ar', str(target_sr),    # 设置采样率为 target_sr
        '-acodec', 'pcm_s16le',   # 设置音频编码为 16-bit PCM (标准 WAV)
        '-loglevel', 'error',     # 只输出错误信息
        output_path              # 输出文件
    ]
    try:
        result = subprocess.run(cmd, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True,
                                encoding='utf-8')
        if os.path.exists(output_path):
            print(f"转换成功: '{os.path.basename(output_path)}'")
            return True
        else:
            print(f"转换失败, 未生成输出文件。FFmpeg stderr: {result.stderr}")
            return False
    except subprocess.CalledProcessError as e:
        print(f"FFmpeg 转换失败。")
        print(f"  命令: {' '.join(cmd)}")
        print(f"  返回码: {e.returncode}")
        print(f"  错误输出: {e.stderr}")
        return False
    except Exception as e:
        print(f"执行 FFmpeg 时发生未知错误: {e}")
        return False

# ----- 音频加载函数 -----
def load_audio(filepath, target_sr=16000):
    """
    加载音频, 确保为 mono 和 target_sr 采样率。
    假定输入已经是 WAV 格式。
    """
    try:
        # sr=None 意味着加载原始采样率, 后面再处理
        waveform, sr = librosa.load(filepath, sr=None, mono=False) # 先不强制 mono

        # 1. 转为 Mono
        if waveform.ndim > 1:
            waveform = librosa.to_mono(waveform)

        # 2. 重采样到目标采样率
        if sr != target_sr:
            print(f"resampling from {sr} Hz to {target_sr} Hz...")
            waveform = librosa.resample(waveform, orig_sr=sr, target_sr=target_sr)

        print(f"音频加载成功, 长度: {len(waveform)/target_sr:.2f} 秒")
        return waveform
    except Exception as e:
        print(f"音频加载失败: {filepath}")
        print(f"  错误信息: {e}")
        return None

# ----- 主分类函数 -----
def classify_audio(filepath, model_url="https://tfhub.dev/google/yamnet/1", score_threshold=0.1, top_n=5):
    """
    使用 YAMNet 分类音频, 并判断是语音还是音乐。
    """
```

```

Args:
    filepath (str): 输入音频文件的路径。
    model_url (str): YAMNet 模型的 TensorFlow Hub URL。
    score_threshold (float): 判断类别是否存在的最低平均得分。
    top_n (int): 显示得分最高的 N 个标签。
"""
print("-" * 30)
print(f" 开始处理文件: {filepath}")

# Step 0: 检查 FFmpeg (只在脚本开始时检查一次)
# (在 main 函数里检查)

# Step 1: 转换格式为临时 WAV 文件
temp_wav_file = None
try:
    # 创建一个临时的 WAV 文件名
    with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp_f:
        temp_wav_path = tmp_f.name
        temp_wav_file = temp_wav_path # 保存路径以便后续删除

    if not convert_to_wav(filepath, temp_wav_path):
        return # 转换失败, 退出函数

    # Step 2: 加载转换后的音频
    print(" 正在加载音频...")
    waveform = load_audio(temp_wav_path)
    if waveform is None:
        return # 加载失败, 退出函数

    # Step 3: 加载 YAMNet 模型和标签
    print("正在加载 YAMNet 模型...")
    try:
        model = hub.load(model_url)
        # 加载标签映射表 (确保能访问网络或已缓存)
        class_map_path = model.class_map_path().numpy().decode('utf-8')
        class_map_df = pd.read_csv(class_map_path)
        class_names = class_map_df['display_name'].tolist()
        print(" 模型和标签加载完成。")
    except Exception as e:
        print(f" 加载 YAMNet 模型或标签失败: {e}")
        return

    # Step 4: 分类预测
    print(" Keras 模型已加载, 正在使用...")
    # YAMNet 模型期望输入是 [-1, 1] 范围内的 float32 NumPy 数组
    # librosa 加载的就是 float32, 范围通常在 [-1, 1] 内, 无需额外缩放
    scores, embeddings, spectrogram = model(waveform)

    # 计算整个片段的平均得分
    mean_scores = tf.reduce_mean(scores, axis=0)
    mean_scores_np = mean_scores.numpy()

    # 获取得分最高的 Top N 标签
    top_n_indices = np.argsort(mean_scores_np)[::-1][:top_n]

    print(f"\n Top {top_n} 预测标签:")
    for i in top_n_indices:
        print(f"   - {class_names[i]}: {mean_scores_np[i]:.3f}")

    # Step 5: 判断是语音还是音乐 (改进版逻辑)
    print("\n 分析语音/音乐成分...")
    speech_indices = [i for i, name in enumerate(class_names) if 'speech' in name.lower()]
    music_indices = [i for i, name in enumerate(class_names)
                     if 'music' in name.lower() or 'singing' in name.lower() or 'musical instrument' in name.lower()]

    # 计算相关类别的总平均得分
    speech_score = np.sum(mean_scores_np[speech_indices])
    music_score = np.sum(mean_scores_np[music_indices])

    print(f"   - 语音相关总得分: {speech_score:.3f}")
    print(f"   - 音乐相关总得分: {music_score:.3f}")
    print(f"   - 判断阈值: {score_threshold}")

    is_speech = speech_score >= score_threshold
    is_music = music_score >= score_threshold

    print("\n 最终识别结果:")
    if is_speech and is_music:
        if speech_score > music_score:

```



```

        print("    主要为【语音】(混合了音乐成分)")
    elif music_score > speech_score:
        print("    主要为【音乐】(混合了语音成分)")
    else:
        print("    【混合】(语音和音乐成分相当)")
    elif is_speech:
        print("    【语音】")
    elif is_music:
        print("    【音乐】")
    else:
        print("    【其他/不确定】(语音和音乐得分均低于阈值)")
        print(f"    (最可能的标签是: {class_names[top_n_indices[0]]} - {mean_scores_np[top_n_indices[0]].3f})")

except Exception as e:
    print(f"  处理过程中发生错误: {e}")
finally:
    # Step 6: 清理临时文件
    if temp_wav_file and os.path.exists(temp_wav_file):
        try:
            os.remove(temp_wav_file)
            print(f"\n  已删除临时文件: {temp_wav_file}")
        except OSError as e:
            print(f"  删除临时文件失败: {e}")
    print("-" * 30)

# ----- 命令行执行入口 -----
if __name__ == "__main__":
    # 首先检查 FFmpeg 是否存在
    if not check_ffmpeg():
        exit(1) # 如果 FFmpeg 不可用, 则退出

    parser = argparse.ArgumentParser(description='使用 YAMNet 模型识别音频是语音还是音乐。')
    parser.add_argument('input_file', type=str, help='要分析的输入音频文件路径 (任何 ffmpeg 支持的格式)。')
    parser.add_argument('--threshold', type=float, default=0.1, help='判断语音/音乐类别是否存在的得分阈值 (默认: 0.1)。')
    parser.add_argument('--top_n', type=int, default=5, help='显示得分最高的 N 个标签 (默认: 5)。')

    args = parser.parse_args()

    if not os.path.exists(args.input_file):
        print(f"  错误: 输入文件未找到 - {args.input_file}")
        exit(1)

    # 调用主分类函数
    classify_audio(args.input_file, score_threshold=args.threshold, top_n=args.top_n)

#####3.2 自适应编码方案-频谱特点和动态范围识别模块设计.py
import os
import numpy as np
import librosa
from scipy.stats import kurtosis, skew

# 音频分析类
class AudioAnalyzer:
    def __init__(self, audio_file):
        self.audio_file = audio_file
        self.audio_data, self.sr = librosa.load(audio_file, sr=None)
        self.duration = librosa.get_duration(y=self.audio_data, sr=self.sr)
        self.channels = 1 if len(self.audio_data.shape) == 1 else self.audio_data.shape[0] # 通道数
        self.sample_width = 16 # 默认 16 位深度, 若需要可以改成动态计算
        self.bit_depth = self.sample_width

    def calculate_zero_crossings(self):
        return np.mean(librosa.feature.zero_crossing_rate(self.audio_data))

    def calculate_rms(self):
        return np.mean(librosa.feature.rms(y=self.audio_data))

    def calculate_spectral_centroid(self):
        # 计算 STFT 幅度谱
        D = np.abs(librosa.stft(self.audio_data))
        # 计算谱质心 (不使用对数变换, 避免负值)
        return np.mean(librosa.feature.spectral_centroid(S=D, sr=self.sr))

    def calculate_spectral_flatness(self):
        return np.mean(librosa.feature.spectral_flatness(y=self.audio_data))

    def calculate_mfcc(self):
        mfcc = librosa.feature.mfcc(y=self.audio_data, sr=self.sr, n_mfcc=13)
        return np.mean(mfcc, axis=1)

```

```

def calculate_spectral_entropy(self):
    D = np.abs(librosa.stft(self.audio_data))
    D_db = librosa.amplitude_to_db(D, ref=np.max)
    D_db = np.maximum(D_db, 1e-10) # 避免对数中的零值
    return -np.sum(D_db * np.log(D_db)) / len(D_db)

def calculate_skewness(self):
    return skew(self.audio_data)

def calculate_kurtosis(self):
    return kurtosis(self.audio_data)

def calculate_dynamic_range(self):
    rms = librosa.feature.rms(y=self.audio_data)
    return np.max(rms) - np.min(rms)

def calculate_pitch(self):
    pitches, _ = librosa.core.piptrack(y=self.audio_data, sr=self.sr)
    return np.mean(pitches)

def calculate_loudness(self):
    return np.mean(librosa.feature.rms(y=self.audio_data))

def calculate_snr(self):
    return self.perceptual_snr(self.audio_data, self.audio_data) # 使用相同的音频作为参考和测试

def get_audio_info(self):
    zero_crossings = self.calculate_zero_crossings()
    rms = self.calculate_rms()
    spectral_centroid = self.calculate_spectral_centroid()
    spectral_flatness = self.calculate_spectral_flatness()
    mfcc = self.calculate_mfcc()
    spectral_entropy = self.calculate_spectral_entropy()
    skewness = self.calculate_skewness()
    kurtosis_value = self.calculate_kurtosis()
    dynamic_range = self.calculate_dynamic_range()
    pitch = self.calculate_pitch()
    loudness = self.calculate_loudness()
    snr = self.calculate_snr()

    return {
        '时长 (s)': self.duration,
        '采样率 (Hz)': self.sr,
        '通道数': self.channels,
        '比特深度': self.bit_depth,
        '零交叉率 (平均)': zero_crossings,
        '均方根 (RMS)': rms,
        '谱质心 (Spectral Centroid)': spectral_centroid,
        '谱平坦度 (Spectral Flatness)': spectral_flatness,
        'MFCC (均值)': mfcc.tolist(),
        '香农熵 (Spectral Entropy)': spectral_entropy,
        '偏度 (Skewness)': skewness,
        '峰度 (Kurtosis)': kurtosis_value,
        '动态范围 (RMS)': dynamic_range,
        '音高 (Pitch)': pitch,
        '音量 (Loudness)': loudness
    }

# 主程序
if __name__ == "__main__":
    # 输入音频文件路径
    file_path = input("请输入音频文件路径: ")

    if os.path.exists(file_path):
        analyzer = AudioAnalyzer(file_path)

        # 获取音频的分析信息
        audio_info = analyzer.get_audio_info()

        # 打印分析结果
        print(f"\n 音频分析结果: \n")
        print(f"音频时长: {audio_info['时长 (s)']:.2f} 秒")
        print(f"采样率: {audio_info['采样率 (Hz)']} Hz")
        print(f"通道数: {audio_info['通道数']}")
        print(f"比特深度: {audio_info['比特深度']} 位")
        print(f"零交叉率 (平均): {audio_info['零交叉率 (平均)']:.4f}")
        print(f"均方根 (RMS): {audio_info['均方根 (RMS)']:.4f}")
        print(f"谱质心: {audio_info['谱质心 (Spectral Centroid)']:.2f}")
        print(f"谱平坦度: {audio_info['谱平坦度 (Spectral Flatness)']:.3f}")
        print(f"MFCC (均值): {audio_info['MFCC (均值)']}")

```

```

print(f"香农熵: {audio_info['香农熵 (Spectral Entropy)']:.2f}")
print(f"偏度: {audio_info['偏度 (Skewness)']:.2f}")
print(f"峰度: {audio_info['峰度 (Kurtosis)']:.2f}")
print(f"动态范围 (RMS): {audio_info['动态范围 (RMS)']:.4f}")
print(f"音高: {audio_info['音高 (Pitch)']:.2f} Hz")
print(f"音量: {audio_info['音量 (Loudness)']:.4f}")

else:
    print("文件路径无效, 请检查并重新输入。")
#####3.3 音频类型识别模型.py
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import joblib
import sys
import os
import itertools
from datetime import datetime

# Excel 数据路径
DATA_PATH = r"F:\DeskTop\MATH\Q3\特征数据集\音频特征_不转码含 SNR_更新版.xlsx"
MODEL_PATH = "audio_model.h5"
SCALER_PATH = "scaler.save"
ENCODER_PATH = "encoders.save"

formats = {
    'wav': {
        'sample_rates': [8000, 16000, 22050, 32000, 44100, 48000],
        'bit_depths': [8, 16, 24, 32],
        'bitrates': [0]
    },
    'mp3': {
        'sample_rates': [22050, 32000, 44100, 48000],
        'bit_depths': [0],
        'bitrates': [64, 96, 128, 160, 192, 256, 320]
    },
    'aac': {
        'sample_rates': [22050, 32000, 44100, 48000],
        'bit_depths': [0],
        'bitrates': [64, 96, 128, 160, 192, 256, 320]
    }
}

all_combinations = [
    (fmt, sr, bd, br)
    for fmt, params in formats.items()
    for sr, bd, br in itertools.product(params['sample_rates'], params['bit_depths'], params['bitrates'])
]

def train_and_save():
    df = pd.read_excel(DATA_PATH)
    df['音频类型'] = df['文件名'].apply(lambda x: '音乐' if '音乐' in x or 'Music' in x else '语音')
    df['音频格式'] = df['文件名'].apply(lambda x: os.path.splitext(x)[-1][1:])

    le_format = LabelEncoder()
    df['音频格式编码'] = le_format.fit_transform(df['音频格式'])
    le_type = LabelEncoder()
    df['音频类型编码'] = le_type.fit_transform(df['音频类型'])

    features = [
        '采样率', '通道数', '比特深度', '比特率(kbps)', '零交叉率', 'RMS 能量', '谱质心', '香农熵',
        '偏度', '峰度', '动态范围', '音高', '音量', '音频格式编码', '音频类型编码'
    ]
    target = '性价比'

    df[features] = df[features].fillna(0)

    scaler = StandardScaler()
    X = scaler.fit_transform(df[features])
    y = df[target].values

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = keras.Sequential([
        layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
        layers.Dense(32, activation='relu'),

```



```

import os

# 加载数据 (保持不变)
try:
    df = pd.read_excel("/kaggle/input/finaldata/Final.xlsx")
except FileNotFoundError:
    print("错误: 数据集文件未找到。请检查文件路径是否正确。")
    raise

# 移除原始音频的行 (保持不变)
df_filtered = df[~df['音频类型'].isin(['原始语音', '原始音乐'])].copy()

# 计算性价比 (保持不变)
df_filtered.loc[:, '性价比'] = df_filtered['信噪比'] / df_filtered['文件大小比']

# 打印 DataFrame 的列名以进行检查
print("DataFrame 的列名:", df_filtered.columns)

# 定义特征列和目标列
feature_columns = ['文件大小(KB)', '信噪比', '性价比']
target_rate_column = '采样率'
target_depth_column_wav = '比特深度'
target_rate_column_mp3 = '比特率'
target_rate_column_aac = '比特率'
file_format_column = '文件格式'

# 根据文件格式分割 DataFrame
df_wav = df_filtered[df_filtered['文件格式'] == 'wav'].copy()
df_mp3 = df_filtered[df_filtered['文件格式'] == 'mp3'].copy()
df_aac = df_filtered[df_filtered['文件格式'] == 'aac'].copy()

# 分割特征和目标
X_wav = df_wav[feature_columns]
y_depth_wav = df_wav[target_depth_column_wav]

X_mp3 = df_mp3[feature_columns]
y_rate_mp3 = df_mp3[target_rate_column_mp3]

X_aac = df_aac[feature_columns]
y_rate_aac = df_aac[target_rate_column_aac]

X_all = df_filtered[feature_columns]
y_rate_all = df_filtered[target_rate_column]

# 处理 NaN 值
imputer = SimpleImputer(strategy='mean')
X_imputed_wav = imputer.fit_transform(X_wav) if not X_wav.empty else np.array([])
X_imputed_mp3 = imputer.fit_transform(X_mp3) if not X_mp3.empty else np.array([])
X_imputed_aac = imputer.fit_transform(X_aac) if not X_aac.empty else np.array([])
X_imputed_all = imputer.fit_transform(X_all) if not X_all.empty else np.array([])

# 分割训练集和测试集
X_train_wav, X_test_wav, y_depth_wav_train, y_depth_wav_test = train_test_split(
    X_imputed_wav, y_depth_wav, test_size=0.2, random_state=42
) if X_imputed_wav.shape[0] > 0 else (np.array([]), np.array([]), pd.Series([]), pd.Series([]))

X_train_mp3, X_test_mp3, y_rate_mp3_train, y_rate_mp3_test = train_test_split(
    X_imputed_mp3, y_rate_mp3, test_size=0.2, random_state=42
) if X_imputed_mp3.shape[0] > 0 else (np.array([]), np.array([]), pd.Series([]), pd.Series([]))

X_train_aac, X_test_aac, y_rate_aac_train, y_rate_aac_test = train_test_split(
    X_imputed_aac, y_rate_aac, test_size=0.2, random_state=42
) if X_imputed_aac.shape[0] > 0 else (np.array([]), np.array([]), pd.Series([]), pd.Series([]))

X_train_rate, X_test_rate, y_rate_train, y_rate_test = train_test_split(
    X_imputed_all, y_rate_all, test_size=0.2, random_state=42
) if X_imputed_all.shape[0] > 0 else (np.array([]), np.array([]), pd.Series([]), pd.Series([]))

# --- 模型和参数网格 (针对不同格式分别进行参数调优) ---
best_params = {}
scenarios = ['移动流媒体', '高保真音乐收藏', '通用下载/存储'] # 定义场景列表

for scenario in scenarios:
    print(f"\n--- 针对场景: {scenario} ---")

    # --- 采样率模型 ---
    model_rate = RandomForestRegressor(random_state=42)
    param_grid_rate = {
        'n_estimators': [50, 100],
        'max_depth': [None, 5],
    
```

```

        'min_samples_split': [2, 3],
        'min_samples_leaf': [1, 2]
    }
    grid_search_rate = GridSearchCV(model_rate, param_grid_rate, cv=2, scoring='neg_mean_squared_error', n_jobs=-1)
    grid_search_rate.fit(X_train_rate, y_rate_train)
    best_params[f'{scenario}_rate'] = grid_search_rate.best_params_
    print(f"场景 {scenario} - 采样率最佳参数: {best_params[f'{scenario}_rate']}")

# --- WAV 比特深度模型 ---
if X_train_wav.shape[0] > 0:
    model_depth_wav = RandomForestRegressor(random_state=42)
    param_grid_depth_wav = {
        'n_estimators': [50, 100],
        'max_depth': [None, 5],
        'min_samples_split': [2, 3],
        'min_samples_leaf': [1, 2]
    }
    grid_search_depth_wav = GridSearchCV(model_depth_wav, param_grid_depth_wav, cv=2,
scoring='neg_mean_squared_error', n_jobs=-1)
    grid_search_depth_wav.fit(X_train_wav, y_depth_wav_train)
    best_params[f'{scenario}_depth_wav'] = grid_search_depth_wav.best_params_
    print(f"场景 {scenario} - WAV 比特深度最佳参数: {best_params[f'{scenario}_depth_wav']}")
else:
    best_params[f'{scenario}_depth_wav'] = None
    print(f"场景 {scenario} - 没有 WAV 数据用于训练比特深度模型。")

# --- MP3 比特率模型 ---
if X_train_mp3.shape[0] > 0:
    model_rate_mp3 = RandomForestRegressor(random_state=42)
    param_grid_rate_mp3 = {
        'n_estimators': [50, 100],
        'max_depth': [None, 5],
        'min_samples_split': [2, 3],
        'min_samples_leaf': [1, 2]
    }
    grid_search_rate_mp3 = GridSearchCV(model_rate_mp3, param_grid_rate_mp3, cv=2, scoring='neg_mean_squared_error',
n_jobs=-1)
    grid_search_rate_mp3.fit(X_train_mp3, y_rate_mp3_train)
    best_params[f'{scenario}_rate_mp3'] = grid_search_rate_mp3.best_params_
    print(f"场景 {scenario} - MP3 比特率最佳参数: {best_params[f'{scenario}_rate_mp3']}")
else:
    best_params[f'{scenario}_rate_mp3'] = None
    print(f"场景 {scenario} - 没有 MP3 数据用于训练比特率模型。")

# --- AAC 比特率模型 ---
if X_train_aac.shape[0] > 0:
    model_rate_aac = RandomForestRegressor(random_state=42)
    param_grid_rate_aac = {
        'n_estimators': [50, 100],
        'max_depth': [None, 5],
        'min_samples_split': [2, 3],
        'min_samples_leaf': [1, 2]
    }
    grid_search_rate_aac = GridSearchCV(model_rate_aac, param_grid_rate_aac, cv=2, scoring='neg_mean_squared_error',
n_jobs=-1)
    grid_search_rate_aac.fit(X_train_aac, y_rate_aac_train)
    best_params[f'{scenario}_rate_aac'] = grid_search_rate_aac.best_params_
    print(f"场景 {scenario} - AAC 比特率最佳参数: {best_params[f'{scenario}_rate_aac']}")
else:
    best_params[f'{scenario}_rate_aac'] = None
    print(f"场景 {scenario} - 没有 AAC 数据用于训练比特率模型。")

def extract_audio_features_simplified(audio_path):
    try:
        y, sr = librosa.load(audio_path)
        amplitude = np.mean(np.abs(y))
        snr = 0 # 简化, 假设为 0
        file_size_kb = os.path.getsize(audio_path) / 1024 if os.path.exists(audio_path) else 0
        file_size_ratio = 1 # 简化, 假设为 1
        cost_effectiveness = snr / (file_size_ratio + 1e-9) if file_size_ratio != 0 else snr
        file_format = audio_path.split('.')[-1].lower() if os.path.exists(audio_path) else None

        return pd.Series({
            '文件大小(KB)': file_size_kb,
            '信噪比': snr,
            '性价比': cost_effectiveness,
            '文件格式': file_format
        })
    except Exception as e:
        print(f"处理音频文件时出错: {e}")

```



```

        return None

def recommend_audio_parameters_weighted_all_formats(audio_path, audio_type_str, scenario, best_params):
    extracted_features = extract_audio_features_simplified(audio_path)
    if extracted_features is None:
        return None

    recommendations = {'音频类型': audio_type_str.lower(), '使用场景': scenario}
    all_format_recommendations = {}

    for format_to_recommend in ['wav', 'mp3', 'aac']:
        relevant_df = df_filtered[df_filtered['文件格式'] == format_to_recommend].copy()
        if relevant_df.empty:
            all_format_recommendations[format_to_recommend] = f"没有 {format_to_recommend} 格式的有效数据用于推荐。"
            continue

        # 计算性价比
        relevant_df.loc[:, '性价比'] = relevant_df['信噪比'] / (relevant_df['文件大小'] + 1e-9)

        # 定义权重
        weight_file_size = 0
        weight_snr = 0
        weight_cost = 0

        if scenario == '移动流媒体':
            weight_file_size = 0.45
            weight_snr = 0.25
            weight_cost = 0.3
        elif scenario == '高保真音乐收藏':
            weight_file_size = 0.1
            weight_snr = 0.8
            weight_cost = 0.1
        elif scenario == '通用下载/存储':
            weight_file_size = 0.35
            weight_snr = 0.35
            weight_cost = 0.3

        # 归一化特征, 避免不同尺度影响
        if not relevant_df['文件大小(KB)'].empty and relevant_df['文件大小(KB)'].max() > relevant_df['文件大小(KB)'].min():
            relevant_df['文件大小(KB)_norm'] = (relevant_df['文件大小(KB)'] - relevant_df['文件大小(KB)'].min()) / \
            (relevant_df['文件大小(KB)'].max() - relevant_df['文件大小(KB)'].min() + 1e-9)
        else:
            relevant_df['文件大小(KB)_norm'] = 0.5 # 避免除以零

        if not relevant_df['信噪比'].empty and relevant_df['信噪比'].max() > relevant_df['信噪比'].min():
            relevant_df['信噪比_norm'] = (relevant_df['信噪比'] - relevant_df['信噪比'].min()) / (relevant_df['信噪比'].max() - relevant_df['信噪比'].min() + 1e-9)
        else:
            relevant_df['信噪比_norm'] = 0.5

        if not relevant_df['性价比'].empty and relevant_df['性价比'].max() > relevant_df['性价比'].min():
            relevant_df['性价比_norm'] = (relevant_df['性价比'] - relevant_df['性价比'].min()) / (relevant_df['性价比'].max() - relevant_df['性价比'].min() + 1e-9)
        else:
            relevant_df['性价比_norm'] = 0.5

        # 计算加权得分
        relevant_df['加权得分'] = (1 - relevant_df['文件大小(KB)_norm']) * weight_file_size + \
            relevant_df['信噪比_norm'] * weight_snr + \
            relevant_df['性价比_norm'] * weight_cost

        relevant_df_sorted = relevant_df.sort_values(by='加权得分', ascending=False)

        if not relevant_df_sorted.empty:
            best_row = relevant_df_sorted.iloc[0]
            all_format_recommendations[format_to_recommend] = {
                '推荐采样率': int(best_row['采样率']),
                '推荐通道数': int(best_row['通道数']),
                '推荐比特深度': int(best_row['比特深度']) if format_to_recommend == 'wav' else None,
                '推荐比特率': int(best_row['比特率']) if format_to_recommend in ['mp3', 'aac'] else None,
                '预估文件大小(KB)': best_row['文件大小(KB)'],
                '预估信噪比': best_row['信噪比'],
                '预估性价比': best_row['性价比']
            }
        else:
            all_format_recommendations[format_to_recommend] = f"没有 {format_to_recommend} 格式的有效数据用于推荐。"

    recommendations['各格式推荐'] = all_format_recommendations
    return recommendations

```

```

if __name__ == "__main__":
    test_audio_file = "/kaggle/input/myspeech/_48kHz_24bit.wav" # 替换为您的实际音频文件路径
    user_provided_type = "音乐" # 或者 "语音"

    if not os.path.exists(test_audio_file):
        print(f"错误: 测试音频文件 {test_audio_file} 未找到。请确保文件已上传到正确的路径。")
    else:
        for scenario in scenarios:
            recommended_parameters = recommend_audio_parameters_weighted_all_formats(
                test_audio_file, user_provided_type, scenario, {} # best_params 在这个简化版本中未使用
            )

            if recommended_parameters:
                print(f"\n--- {scenario} 场景下的推荐 ---")
                print("为音频文件:", test_audio_file)
                print("用户提供的音频类型:", user_provided_type)
                if '各格式推荐' in recommended_parameters:
                    for format, recs in recommended_parameters['各格式推荐'].items():
                        print(f"\n --- {format.upper()} 格式推荐 ---")
                        if isinstance(recs, str):
                            print(f" - {recs}")
                        else:
                            for key, value in recs.items():
                                if value is not None:
                                    print(f" - {key}: {value}")                                print(f" - {key}: {value}")

#####问题四#####
#####基于规则的噪声识别.py-
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import librosa
import csv
import os
import warnings
import io # 用于在内存中处理字符串 IO
import urllib.request # 用于发出网络请求
from urllib.error import URLError # 用于捕获网络错误

# 抑制警告
warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore', category=UserWarning)

# -----
# 全局变量与模型加载
# -----
# YAMNet 模型句柄
YAMNET_MODEL_HANDLE = 'https://tfhub.dev/google/yamnet/1'
# YAMNet 类别映射文件的在线 URL (来自 TensorFlow Models GitHub 仓库)
# *** 注意: 这个 URL 可能会变化, 如果失效需要更新 ***
YAMNET_CLASS_MAP_URL =
'https://raw.githubusercontent.com/tensorflow/models/master/research/audioset/yamnet/yamnet_class_map.csv'
# YAMNet 要求的采样率
TARGET_SR = 16000

# --- 加载 YAMNet 模型 ---
yamnet_model = None
try:
    print("正在加载 YAMNet 模型...")
    yamnet_model = hub.load(YAMNET_MODEL_HANDLE)
    print("YAMNet 模型加载成功。")
except Exception as e:
    print(f"加载 YAMNet 模型失败: {e}")
    print("请检查 TensorFlow Hub 连接或模型句柄。")

# --- 在线加载 YAMNet 类别名称 ---
class_names = []
if yamnet_model is not None:
    print(f"正在尝试从 URL 在线加载 YAMNet 类别映射: {YAMNET_CLASS_MAP_URL}")
    try:
        # 发起网络请求获取 CSV 文件内容
        with urllib.request.urlopen(YAMNET_CLASS_MAP_URL, timeout=10) as response: # 设置 10 秒超时
            # 读取内容并解码为 UTF-8 字符串
            csv_data = response.read().decode('utf-8')
            # 使用 io.StringIO 将字符串模拟成文件供 csv.reader 读取
            csvfile = io.StringIO(csv_data)
            reader = csv.reader(csvfile)
            next(reader) # 跳过表头 (通常是 index, mid, display_name)
            for row in reader:
                # 假设 display_name 在第三列 (索引为 2)
                if len(row) > 2:

```

```

        class_names.append(row[2])
    else:
        print(f"警告: CSV 文件行格式不符: {row}") # 打印问题行
if class_names:
    print(f"成功在线加载 {len(class_names)} 个 YAMNet 类别名称。")
else:
    print("警告: 在线加载的类别列表为空, 请检查 URL 或文件格式。")

except URLError as e:
    print(f"错误: 无法访问 YAMNet 类别映射 URL: {e}")
    print("请检查网络连接或 URL 是否仍然有效。")
except Exception as e:
    print(f"在线加载或解析 YAMNet 类别映射时发生未知错误: {e}")

if yamnet_model is not None and not class_names:
    print("警告: 未能加载 YAMNet 类别名称, 预测结果将只显示索引。")

# -----
# 函数 1: 加载和预处理音频 (与之前代码相同)
# -----
def load_audio_for_yamnet(audio_path, target_sr=TARGET_SR):
    """加载音频, 转换为单声道, 重采样到目标 SR, 并确保值在 -1 到 1 之间。"""
    try:
        wav_data, sr = librosa.load(audio_path, sr=target_sr, mono=True)
        max_amp = np.max(np.abs(wav_data))
        if max_amp > 1.0: wav_data = wav_data / max_amp
        elif max_amp == 0: print(f"警告: 音频文件 {os.path.basename(audio_path)} 是静音的。")
        return wav_data, target_sr
    except Exception as e:
        print(f"加载或处理音频 {os.path.basename(audio_path)} 时出错: {e}")
        return None, None

# -----
# 函数 2: 运行 YAMNet 推理并获取预测结果 (修改版)
# -----
def get_yamnet_predictions(model, wav_data, class_names_list=None, top_n=10):
    """
    对加载的音频数据运行 YAMNet 模型, 并返回 Top N 预测结果。
    (修改为直接调用模型)

    Args:
        model: 已加载的 YAMNet TensorFlow Hub 模型。
        wav_data (np.ndarray): 符合 YAMNet 输入要求的音频波形数据。
        class_names_list (list, optional): YAMNet 类别名称列表。如果提供, 结果包含名称; 否则仅索引。
        top_n (int): 返回得分最高的类别数量。

    Returns:
        list: 一个包含元组 (label, score) 的列表, 按分数降序排列。
            如果 class_names_list 未提供, label 将是类别索引。
            如果出错则返回 None。
    """
    if model is None or wav_data is None:
        print("错误: 模型未加载或音频数据无效。")
        return None

    try:
        # --- 修改点: 不再获取 'default' 签名, 直接调用模型 ---
        # 检查模型是否可用
        if not callable(model):
            print("错误: 加载的 YAMNet 模型对象不可直接调用。")
            # 可以尝试打印 model 的可用方法或属性来调试: print(dir(model))
            return None

        # 直接调用模型进行推理
        # YAMNet 模型期望输入是 tf.float32 类型, 长度至少为 15600 个样本 (约 0.975 秒)
        # 如果音频太短, 可能需要填充或处理
        if len(wav_data) < 15600:
            print(f"警告: 音频长度 {len(wav_data)} 小于 YAMNet 推荐的最小长度 15600。结果可能不准确。")
            # 可以选择填充: np.pad(wav_data, (0, 15600 - len(wav_data)))
            # 或者直接处理, 但这里我们先不填充, 如果报错再考虑

        scores, embeddings, spectrogram = model(tf.constant(wav_data, dtype=tf.float32))
        # -----

        # scores 的形状是 (N, 521), N 是音频被分割成的帧数
        scores_np = scores.numpy()

        # 分析策略: 取所有帧中每个类别的平均得分
        mean_scores = np.mean(scores_np, axis=0)

```

```

# 获取平均分数最高的 N 个类别的索引
top_n_indices = np.argsort(mean_scores)[-top_n:][::-1] # 从高到低排序

predictions = []
for i in top_n_indices:
    score = mean_scores[i]
    # 如果类别名称列表可用, 则使用名称; 否则使用索引
    label = class_names_list[i] if class_names_list and i < len(class_names_list) else f"Index_{i}"
    predictions.append((label, score))

return predictions

except KeyError as ke:
    print(f"运行 YAMNet 推理时发生 KeyError: {ke}")
    print("这可能意味着模型的接口与预期不符。尝试检查模型的属性和方法。")
    # print(dir(model)) # 打印模型可用属性/方法
    return None
except Exception as e:
    print(f"运行 YAMNet 推理时发生未知错误: {e}")
    # import traceback # 取消注释以查看详细错误
    # traceback.print_exc()
    return None

def plot_prediction_time_series(scores_tensor, class_names_list, selected_tags, title="Prediction over Time"):
    """
    可视化指定标签在时间上的概率变化曲线。

    Args:
        scores_tensor (tf.Tensor): 模型输出的每帧 scores, 形状为 (N, 521)。
        class_names_list (list): 标签名称列表。
        selected_tags (list): 要绘图的类别名列表 (如 ['White noise', 'Click'])。
        title (str): 图表标题。
    """
    import matplotlib.pyplot as plt

    scores_np = scores_tensor.numpy()
    timestamps = np.arange(scores_np.shape[0]) * 0.96 # 每帧时间长度约为 0.96 秒

    plt.figure(figsize=(12, 6))
    for tag in selected_tags:
        if tag in class_names_list:
            idx = class_names_list.index(tag)
            plt.plot(timestamps, scores_np[:, idx], label=tag)
        else:
            print(f"警告: 标签 '{tag}' 不在类别列表中, 跳过绘制。")

    plt.xlabel('Time (s)')
    plt.ylabel('Probability')
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# -----
# 主程序入口 (与之前类似)
# -----
if __name__ == "__main__":

    # <<< --- 请将下面两个路径替换为您的实际音频文件路径 --- >>>
    audio_file_1 = 'D:/pythonProject/mathercup/pro4/附件 2/part1.wav'
    audio_file_2 = 'D:/pythonProject/mathercup/pro4/附件 2/part2.wav'

    # 检查模型是否加载成功
    if yamnet_model is None:
        print("YAMNet 模型未成功加载, 无法进行预测。请检查之前的错误信息。")
        exit()

    # --- 处理第一个文件 ---
    print(f"\n--- 正在处理文件 1: {os.path.basename(audio_file_1)} ---")
    wav_data1, sr1 = load_audio_for_yamnet(audio_file_1)
    if wav_data1 is not None:
        # 使用在线加载的 class_names 列表
        predictions1 = get_yamnet_predictions(yamnet_model, wav_data1, class_names, top_n=10)
        print(f"\n文件 '{os.path.basename(audio_file_1)}' 的 YAMNet Top 10 平均预测:")
        if predictions1:
            for label, score in predictions1:

```

```

        print(f"- {label}: {score:.4f}")
    else:
        print("无法获取预测结果。")

# --- 处理第二个文件 ---
print(f"\n--- 正在处理文件 2: {os.path.basename(audio_file_2)} ---")
wav_data2, sr2 = load_audio_for_yamnet(audio_file_2)
if wav_data2 is not None:
    # 使用在线加载的 class_names 列表
    predictions2 = get_yamnet_predictions(yamnet_model, wav_data2, class_names, top_n=10)
    print(f"\n文件 '{os.path.basename(audio_file_2)}' 的 YAMNet Top 10 平均预测:")
    if predictions2:
        for label, score in predictions2:
            print(f"- {label}: {score:.4f}")
    else:
        print("无法获取预测结果。")

#####问题四#####
#####自适应降噪方案.py
# -----
# 导入必要的库
# -----
import numpy as np
import librosa
import scipy.signal
import scipy.stats
import soundfile as sf
import os
import pandas as pd
import warnings
import pywt
import noisereduce as nr
from numpy.lib.stride_tricks import as_strided
import itertools
import random
import time
# import matplotlib.pyplot as plt # <--- 移除绘图库导入

# -----
# 常量与配置
# -----
# 抑制警告 (根据需要调整)
warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore', category=UserWarning)
warnings.filterwarnings('ignore', category=RuntimeWarning)
warnings.filterwarnings("ignore", message="PySoundFile failed. Trying audioread instead.")
warnings.filterwarnings("ignore", message="Trying to estimate tuning from empty frequency set.") # Librosa warning

# -----
# 函数: 噪声成分检测 (基于规则)
# -----
def detect_noise_components(features):
    """根据提取的音频特征检测噪声成分。"""
    if not features: return ['错误: 未提供特征']
    detected_components = set()
    # --- 阈值定义 ---
    kurtosis_impulsive_thresh = 15.0
    energy_peak_rate_impulsive_thresh = 3.0
    spectral_flatness_tonal_thresh = 0.01
    spectral_entropy_tonal_thresh = 4.5
    num_spectral_peaks_tonal_thresh = 50
    harmonic_detected_is_evidence = True
    log_energy_variance_stable_thresh = 3.0

    # --- 突发噪声检测 ---
    kurt_val = features.get('kurtosis_max', 3.0)
    peak_rate_val = features.get('energy_peak_rate', 0)
    if (kurt_val > kurtosis_impulsive_thresh or
        peak_rate_val > energy_peak_rate_impulsive_thresh):
        detected_components.add('Impulsive')

    # --- 带状噪声检测 ---
    flatness = features.get('spectral_flatness_mean', 1.0)
    entropy = features.get('spectral_entropy_mean', 10.0)
    num_peaks = features.get('num_spectral_peaks', 0)
    is_harmonic = features.get('harmonic_detected', False)
    if ((flatness < spectral_flatness_tonal_thresh) or
        (not np.isnan(entropy) and entropy < spectral_entropy_tonal_thresh) or
        (num_peaks >= num_spectral_peaks_tonal_thresh) or
        (is_harmonic and harmonic_detected_is_evidence)):
        detected_components.add('Tonal')

```

```

# --- 最终判断 & 背景噪声 ---
final_components = list(detected_components)
log_energy_var = features.get('log_energy_variance', 5.0)
is_stable = log_energy_var < log_energy_variance_stable_thresh
has_explicit_noise = any(comp in ['Impulsive', 'Tonal'] for comp in final_components)

if has_explicit_noise:
    final_components.append('Background')
    return sorted(list(set(final_components)))
elif is_stable:
    return ['Background']
else:
    return ['Components Uncertain']

# -----
# 模块 1: 突发噪声抑制 (使用小波阈值)
# -----
def remove_impulsive_noise_wavelet(y, sr, features, wavelet='db8', level=None, threshold_mode='soft',
threshold_type='universal'):
    """使用小波阈值法抑制突发噪声。"""
    # print(f" - 应用突发噪声抑制 (小波阈值)...") # 可选的打印信息
    if level is None:
        try:
            wavelet_obj = pywt.Wavelet(wavelet)
            max_level_theor = pywt.dwt_max_level(len(y), wavelet_obj.dec_len)
            if len(y) > (wavelet_obj.dec_len + 1):
                max_practical_level = int(np.log2(len(y) / (wavelet_obj.dec_len + 1)))
            else:
                max_practical_level = 0
            level = min(max_level_theor, max_practical_level, 8) # 加入经验上限 8
        except ValueError:
            return y # 信号太短或 wavelet 名称错误
    if level <= 0:
        return y

    try:
        coeffs = pywt.wavedec(y, wavelet, level=level)
    except ValueError:
        return y # 分解失败

    detail_coeffs = coeffs[-1]
    sigma = np.median(np.abs(detail_coeffs - np.median(detail_coeffs))) / 0.6745 if len(detail_coeffs) > 0 else 0
    threshold = sigma * np.sqrt(2 * np.log(len(y))) if sigma > 0 and len(y) > 1 else 0

    coeffs_thresh = [coeffs[0]]
    for i in range(1, level + 1):
        coeffs_thresh.append(pywt.threshold(coeffs[i], value=threshold, mode=threshold_mode))

    try:
        y_denoised = pywt.waverec(coeffs_thresh, wavelet)
    except ValueError:
        return y # 重构失败

    # 确保长度一致
    if len(y_denoised) != len(y):
        min_len = min(len(y_denoised), len(y))
        y_denoised = y_denoised[:min_len]
        y_original_padded = np.pad(y[:min_len], (0, len(y_denoised)-min_len), mode='constant') # 填充原始信号以匹配
        # 或者简单截断/填充处理后的信号
        if len(y_denoised) < len(y):
            y_denoised = np.pad(y_denoised, (0, len(y) - len(y_denoised)), mode='edge')
        elif len(y_denoised) > len(y):
            y_denoised = y_denoised[:len(y)]

    return y_denoised

# -----
# 模块 2: 带状噪声消除 (使用多个陷波滤波器 - 修正峰值检测)
# -----
def remove_tonal_noise_notch_revised(y, sr, n_fft=2048, hop_length=512, peak_distance_hz=25, peak_prominence_factor=0.1,
quality_factor=50.0, max_notches=15):
    """使用陷波滤波器消除检测到的带状噪声。"""
    # print(f" - 应用带状噪声消除 (陷波滤波器)...") # 可选
    y_processed = y.copy()
    epsilon = 1e-10
    try:
        S_complex = librosa.stft(y_processed, n_fft=n_fft, hop_length=hop_length)
        avg_psd = np.mean(np.abs(S_complex)**2, axis=1)
        freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)

```



```

except Exception:
    return y_processed # STFT 计算失败

peak_distance_bins = max(1, int(peak_distance_hz * n_fft / sr))
median_psd = np.median(avg_psd[avg_psd > epsilon])
tonal_freqs_to_remove = []

if median_psd > epsilon:
    prominence_threshold = peak_prominence_factor * median_psd
    significant_peaks_indices, properties = scipy.signal.find_peaks(
        avg_psd, prominence=prominence_threshold, distance=peak_distance_bins)

    if len(significant_peaks_indices) > 0:
        sorted_indices = significant_peaks_indices[np.argsort(properties['prominences'])[::-1]]
        if len(sorted_indices) > max_notches:
            sorted_indices = sorted_indices[:max_notches]
            tonal_freqs_to_remove = freqs[sorted_indices]
        else:
            return y_processed # 未检测到峰值
    else:
        return y_processed # PSD 过低

for f0 in tonal_freqs_to_remove:
    if f0 <= 0 or f0 >= sr / 2 - 1: continue
    try:
        b, a = scipy.signal.iirnotch(f0, quality_factor, fs=sr)
        zi = scipy.signal.lfilter_zi(b, a) * y_processed[0]
        y_processed, _ = scipy.signal.lfilter(b, a, y_processed, zi=zi)
    except (ValueError, Exception):
        continue # 滤波器设计或应用失败, 跳过此频率
return y_processed

# -----
# 模块 3: 背景噪声抑制 (使用 noisereduce 库)
# -----
def reduce_background_noise_nr(y, sr, features, n_fft=2048, hop_length=512, prop_decrease=0.65,
n_std_thresh_stationary=2.0):
    """使用 noisereduce 库进行背景噪声抑制。"""
    # print(f"    - 应用背景噪声抑制 (noisereduce)...") # 可选
    try:
        y_float = y.astype(np.float32)
        # 尝试兼容新旧 API
        try:
            reduced_noise = nr.reduce_noise(y=y_float, sr=sr, n_fft=n_fft, hop_length=hop_length,
                                            prop_decrease=prop_decrease,
                                            n_std_thresh_stationary=n_std_thresh_stationary,
                                            use_torch=False)

        except TypeError:
            reduced_noise = nr.reduce_noise(audio_clip=y_float, sr=sr, n_fft=n_fft, hop_length=hop_length,
                                            prop_decrease=prop_decrease,
                                            n_std_thresh_stationary=n_std_thresh_stationary,
                                            use_torch=False)

        return reduced_noise
    except ImportError:
        print("    错误: 未安装 'noisereduce'。跳过背景噪声抑制。")
        return y
    except Exception:
        # print(f"    运行 noisereduce 时出错: {e}") # 可选详细错误
        return y # 出错时返回原始信号

# -----
# 函数: 帧生成器 & 估计 SNR (用于参数搜索时的评估)
# -----
def frame_generator(y, frame_length, hop_length):
    """高效生成音频帧。"""
    if y.ndim > 1: y = librosa.to_mono(y)
    n_frames = 1 + (len(y) - frame_length) // hop_length
    if n_frames < 1: return
    shape = (n_frames, frame_length)
    strides = (y.strides[0] * hop_length, y.strides[0])
    frames = as_strided(y, shape=shape, strides=strides)
    yield from frames

def estimate_snr(y, sr, frame_length=2048, hop_length=512, noise_percentile=10):
    """估计音频的信噪比 (SNR)。"""
    epsilon = 1e-10
    try:
        if y.ndim > 1: y = librosa.to_mono(y)
        y = y.astype(np.float32)

```

```

        frame_energies = [np.mean(frame**2) for frame in frame_generator(y, frame_length, hop_length) if len(frame) ==
frame_length]
        if not frame_energies: return np.nan
        frame_energies = np.array(frame_energies)

        if len(frame_energies) < 10:
            noise_power_est = np.mean(frame_energies) if len(frame_energies) > 0 else epsilon
        else:
            energy_threshold = np.percentile(frame_energies, noise_percentile)
            noise_frames_energy = frame_energies[frame_energies <= energy_threshold]
            if len(noise_frames_energy) < max(3, len(frame_energies) * 0.03):
                noise_power_est = np.mean(frame_energies)
            else:
                noise_power_est = np.mean(noise_frames_energy)

        noise_power_est = max(noise_power_est, epsilon)
        total_power = max(np.mean(y**2), epsilon)
        signal_power_est = total_power - noise_power_est

        if signal_power_est <= 0: return -10.0
        snr = signal_power_est / noise_power_est
        snr_db = 10 * np.log10(snr)
        return snr_db
    except Exception:
        return np.nan

# -----
# 函数: 自适应去噪流程
# -----
def adaptive_denoise(y, sr, features, wavelet_type='db8', wavelet_level=None, threshold_mode='soft', tonal_n_fft=2048,
tonal_hop=512, tonal_peak_dist_hz=25, tonal_prom_factor=0.1, tonal_q_factor=50.0, tonal_max_notches=15, bg_n_fft=2048,
bg_hop=512, bg_prop_decrease=0.65, bg_n_std_thresh=2.0):
    """根据检测到的噪声类型应用相应的去噪模块。"""
    # print(f"\n运行自适应去噪...") # 可选
    detected_noises = detect_noise_components(features)
    # print(f" 识别出的噪声成分: {detected_noises}") # 可选

    if '错误' in detected_noises[0] or 'Components Uncertain' in detected_noises[0]:
        return y

    y_processed = y.copy()
    has_processed_impulsive = False
    has_processed_tonal = False
    has_processed_background = False

    # 应用各模块
    if 'Impulsive' in detected_noises:
        y_out = remove_impulsive_noise_wavelet(y_processed, sr, features,
                                                wavelet=wavelet_type, level=wavelet_level,
                                                threshold_mode=threshold_mode)
        if not np.allclose(y_processed, y_out, atol=1e-6):
            y_processed = y_out; has_processed_impulsive = True

    if 'Tonal' in detected_noises:
        y_out = remove_tonal_noise_notch_revised(y_processed, sr,
                                                n_fft=tonal_n_fft, hop_length=tonal_hop,
                                                peak_distance_hz=tonal_peak_dist_hz,
                                                peak_prominence_factor=tonal_prom_factor,
                                                quality_factor=tonal_q_factor,
                                                max_notches=tonal_max_notches)
        if not np.allclose(y_processed, y_out, atol=1e-6):
            y_processed = y_out; has_processed_tonal = True

    run_background_reduction = 'Background' in detected_noises or has_processed_impulsive or has_processed_tonal
    if run_background_reduction:
        y_out = reduce_background_noise_nr(y_processed, sr, features,
                                            n_fft=bg_n_fft, hop_length=bg_hop,
                                            prop_decrease=bg_prop_decrease,
                                            n_std_thresh_stationary=bg_n_std_thresh)
        if not np.allclose(y_processed, y_out, atol=1e-6):
            y_processed = y_out; has_processed_background = True

    # if not (has_processed_impulsive or has_processed_tonal or has_processed_background):
    #     # print(" 注意: 未执行有效的去噪操作。") # 可选

    return y_processed

# -----
# 函数: 参数探索 (基于 SNR) --- 返回所有试验结果
# -----

```

```

def explore_denoise_parameters_snr(y_noisy, sr, features, param_space, search_method='grid', n_trials=20):
    """探索参数空间，寻找最佳估计 SNR，并返回所有试验结果。"""
    best_snr = -np.inf
    best_params = None
    best_y_denoised = None
    trial_results = [] # 用于存储所有试验结果 {'params': {...}, 'snr': float}
    metric = 'snr' # 固定指标

    param_combinations = []
    keys = list(param_space.keys())

    # --- 生成参数组合 ---
    if search_method == 'grid':
        values = [param_space[k] for k in keys]
        param_combinations_tuples = list(itertools.product(*values))
        param_combinations = [dict(zip(keys, combo)) for combo in param_combinations_tuples]
        total_combinations = len(param_combinations)
        print(f"\n 开始网格搜索 (共 {total_combinations} 种组合)，目标指标: {metric.upper()}")
    elif search_method == 'random':
        print(f"\n 开始随机搜索 ({n_trials} 次试验)，目标指标: {metric.upper()}")
        # 固定随机种子以使得随机搜索可复现 (如果需要)
        # SEED = 42
        # np.random.seed(SEED)
        # random.seed(SEED)
        for _ in range(n_trials):
            current_params = {}
            for key, space in param_space.items():
                if isinstance(space, list): current_params[key] = random.choice(space)
                elif isinstance(space, tuple) and len(space) == 3:
                    dist_type, min_val, max_val = space
                    if dist_type == 'uniform': current_params[key] = random.uniform(min_val, max_val)
                    elif dist_type == 'loguniform':
                        if min_val <= 0 or max_val <= 0: current_params[key] = random.uniform(min_val, max_val)
                        else: log_min = np.log(min_val); log_max = np.log(max_val); current_params[key] =
                            np.exp(random.uniform(log_min, log_max))
                    else: current_params[key] = random.uniform(min_val, max_val)
                else: current_params[key] = space # 直接使用
            param_combinations.append(current_params)
        total_combinations = n_trials
    else:
        print(f"错误: 无效的搜索方法 '{search_method}'。")
        return None, -np.inf, None, []

    start_time = time.time()
    print("\n" + "="*30 + "\n!! 警告: 正在基于估计的 SNR 进行参数探索。!!\n!! 最高 SNR 的结果不一定听感最好。      !!\n!! 请
    务必在结束后对最佳 SNR 结果进行听音评估 !!\n!! 并以此参数为起点进行手动微调以获得最佳保真度。 !!\n" + "="*30 + "\n")

    # --- 迭代试验 ---
    for i, current_params in enumerate(param_combinations):
        params_str = ", ".join([f"{k}={v:.3f}" if isinstance(v, float) else f"{k}={v}" for k, v in current_params.items()])
        print(f"--- 试验 {i+1}/{total_combinations}: {params_str} ---")
        current_score = np.nan # 初始化为 NaN
        y_denoised_trial = None
        try:
            y_denoised_trial = adaptive_denoise(y_noisy.copy(), sr, features, **current_params)
            current_score = estimate_snr(y_denoised_trial, sr)
            score_str = f"{current_score:.2f} dB" if not np.isnan(current_score) else "NaN"
            print(f"    -> 估计 SNR = {score_str}")

            if not np.isnan(current_score) and current_score > best_snr:
                best_snr = current_score
                best_params = current_params.copy()
                best_y_denoised = y_denoised_trial.copy() # 只保存最佳的音频
                print(f"    ***** 新的最佳估计 SNR! *****")

        except Exception as e:
            print(f"    处理此参数组合时出错: {e}")
            # import traceback # 可选: 打印详细错误
            # traceback.print_exc(limit=1)

        finally:
            # 记录本次试验结果 (无论成功与否)
            trial_results.append({'params': current_params.copy(), 'snr': current_score})

    end_time = time.time()
    print(f"\n 参数探索完成, 耗时: {end_time - start_time:.2f} 秒。")

    if best_params:

```

```

        best_params_str_final = ", ".join([f"{k}={v:.3f}" if isinstance(v, float) else f"{k}={v}" for k, v in
best_params.items()])
        print(f"找到的最高估计 SNR 参数: {best_params_str_final}")
        print(f"对应的最高估计 SNR: {best_snr:.2f} dB")
        print("\n!! 重要提醒: 请务必听审使用这些参数生成的音频 !!")
    else:
        print(f"\n 未能找到有效的参数组合来优化估计的 {metric.upper()}: ")

    return best_params, best_snr, best_y_denoised, trial_results # 返回所有试验结果

# -----
# 函数: 计算两个音频信号之间的 SNR (原始 vs. 处理后)
# -----
def calculate_snr_between_signals(y_original, y_processed):
    """计算处理后音频相对于原始音频的信噪比 (保真度指标)。"""
    if y_original.shape != y_processed.shape:
        min_len = min(len(y_original), len(y_processed))
        # print(f"警告: calculate_snr_between_signals 收到长度不等的信号, 裁剪至 {min_len}。") # 可选
        y_original = y_original[:min_len]
        y_processed = y_processed[:min_len]
        if min_len == 0: return np.nan

    signal_power = np.mean(y_original**2)
    noise_signal = y_original - y_processed
    noise_power = np.mean(noise_signal**2)
    epsilon = 1e-10
    if signal_power < epsilon: return np.nan
    if noise_power < epsilon: return 100.0 # 差值极小, 视为 SNR 很高
    snr_db = 10 * np.log10(signal_power / noise_power)
    return snr_db

if __name__ == "__main__":

    # --- 配置 ---
    SEARCH_METHOD = 'grid' # 'grid' or 'random'
    N_RANDOM_TRIALS = 30 # 仅当 SEARCH_METHOD = 'random' 时有效
    TARGET_METRIC = 'snr' # 固定为 SNR 用于探索

    # --- 定义参数空间 ---
    parameter_space = {
        'wavelet_type': ['db8'], 'wavelet_level': [None], 'threshold_mode': ['soft'],
        'tonal_n_fft': [2048], 'tonal_hop': [512], 'tonal_peak_dist_hz': [25],
        'tonal_prom_factor': [0.1, 0.2, 0.3],
        'tonal_q_factor': [40.0, 60.0, 80.0],
        'tonal_max_notches': [15],
        'bg_n_fft': [2048], 'bg_hop': [512],
        'bg_prop_decrease': [0.5, 0.65, 0.8],
        'bg_n_std_thresh': [2.0, 2.5, 3.0]
    }
    # 列出所有需要保存到试验结果表中的参数键
    param_keys_for_trial_table = list(parameter_space.keys())

    # --- 加载特征数据 (硬编码示例) ---
    features_part1 = {
        'filename': 'part1.wav', 'duration_seconds': 60.032, 'log_energy_variance': 4.018231,
        'num_energy_peaks': 221, 'energy_peak_rate': 3.68137, 'kurtosis_max': 24.203405,
        'spectral_flatness_mean': 0.002217, 'spectral_entropy_mean': 3.862866,
        'num_spectral_peaks': 100, 'max_peak_prominence': 1765.915, 'harmonic_detected': True
    }
    features_part2 = {
        'filename': 'part2.wav', 'duration_seconds': 60.117333, 'log_energy_variance': 2.965085,
        'num_energy_peaks': 237, 'energy_peak_rate': 3.942291, 'kurtosis_max': 20.303614,
        'spectral_flatness_mean': 0.000616, 'spectral_entropy_mean': 4.206021,
        'num_spectral_peaks': 98, 'max_peak_prominence': 1311.185, 'harmonic_detected': True
    }

    # --- 指定路径 ---
    audio_dir = 'D:/pythonProject/mathercup/pro4/附件 2/' # !! 修改为你的音频目录 !!
    # 更新输出目录名
    output_dir = f'./denoised_audio_{TARGET_METRIC}_exploration_v5_with_improvement/'
    os.makedirs(output_dir, exist_ok=True)

    # --- 文件列表 ---
    all_features_list = [features_part1, features_part2]
    files_to_process = {}
    for feat in all_features_list:
        noisy_filename = feat.get('filename')

```

```

if noisy_filename:
    noisy_path = os.path.join(audio_dir, noisy_filename)
    if os.path.exists(noisy_path): files_to_process[noisy_path] = feat
    else: print(f"警告: 找不到文件 {noisy_path}, 将跳过。")
else: print("警告: 特征字典缺少 'filename'。")

# --- 结果记录 (主结果) ---
results = []

# --- 循环处理 ---
for noisy_path, features in files_to_process.items():
    filename_short = features.get('filename', os.path.basename(noisy_path))
    print(f"\n{'='*60}\n 处理文件: {filename_short}\n{'='*60}")

    y_original, sr_original = None, None # 初始化
    trial_results_current_file = [] # 初始化当前文件的试验结果列表
    snr_original_est = np.nan # 初始化原始 SNR 估计
    best_snr_est = -np.inf # 初始化最佳 SNR 估计
    snr_improvement_est = np.nan # <-- 初始化 SNR 提升幅度

    try:
        # --- 加载原始音频 ---
        y_original, sr_original = librosa.load(noisy_path, sr=None, mono=True)
        print(f" 加载原始音频, sr={sr_original}, duration={librosa.get_duration(y=y_original,
sr=sr_original):.2f}s")

        # --- 估计原始音频的 SNR (参考) ---
        snr_original_est = estimate_snr(y_original, sr_original)
        print(f" 原始音频估计 SNR (内部): {snr_original_est:.2f} dB" if not np.isnan(snr_original_est) else " 无法
估计原始内部 SNR")

        # --- 执行参数探索 (接收 trial_results) ---
        best_params, best_snr_est, best_y_denoised, trial_results_current_file = explore_denoise_parameters_snr(
            y_original, sr_original, features, parameter_space,
            search_method=SEARCH_METHOD, n_trials=N_RANDOM_TRIALS
        )

        # --- 初始化结果变量 ---
        output_path_final = "N/A"
        snr_original_vs_denoised = np.nan # 对比 SNR

        # --- 处理找到的最佳结果 ---
        if best_params and best_y_denoised is not None:
            # 计算原始 vs 去噪后的 SNR (保真度)
            snr_original_vs_denoised = calculate_snr_between_signals(y_original, best_y_denoised)
            if not np.isnan(snr_original_vs_denoised):
                print(f"\n 原始音频 vs 最佳去噪结果 SNR (保真度): {snr_original_vs_denoised:.2f} dB")
                # print(" (此值越高, 表示去噪结果与原始信号越相似, 失真越小)") # 可选解释

            # !! 新增: 计算估计 SNR 的提升 !!
            if not np.isnan(snr_original_est) and best_snr_est > -np.inf:
                snr_improvement_est = best_snr_est - snr_original_est
                print(f" 最佳估计 SNR 提升幅度: {snr_improvement_est:.2f} dB")
            else:
                print(" 无法计算估计 SNR 提升幅度 (原始或最佳 SNR 无效)。")

            # 保存最佳 SNR 对应的音频
            base_name = os.path.basename(noisy_path)
            output_filename = f"{os.path.splitext(base_name)[0]}_denoised_bestEstSNR.wav"
            output_path_final = os.path.join(output_dir, output_filename)
            try:
                sf.write(output_path_final, best_y_denoised, sr_original)
                print(f" 对应最高估计 SNR ({best_snr_est:.2f} dB) 的音频已保存至: {output_path_final}")
            except Exception as e:
                print(f" 保存文件 {output_path_final} 时出错: {e}")
                output_path_final = "Save Error"
        else:
            print(f"\n 未能为文件 {filename_short} 找到有效的最佳估计 SNR 参数组合。")

        # --- 保存当前文件的参数试验结果到单独的 CSV ---
        if trial_results_current_file:
            processed_trials_data = []
            for i, trial in enumerate(trial_results_current_file):
                row_data = {'trial_number': i + 1}
                for key in param_keys_for_trial_table:
                    row_data[key] = trial['params'].get(key, np.nan)
                row_data['estimated_snr'] = trial['snr']

```

```

        processed_trials_data.append(row_data)
    trials_df = pd.DataFrame(processed_trials_data)
    base_name = os.path.splitext(filename_short)[0]
    trials_csv_filename = f"{base_name}_parameter_trials_results.csv"
    trials_csv_path = os.path.join(output_dir, trials_csv_filename)
    try:
        trial_cols_order = ['trial_number'] + param_keys_for_trial_table + ['estimated_snr']
        trial_cols_existing = [c for c in trial_cols_order if c in trials_df.columns]
        trials_df[trial_cols_existing].to_csv(trials_csv_path, index=False, float_format='%.3f')
        print(f"  参数试验结果已保存至: {trials_csv_path}")
    except Exception as e:
        print(f"  保存参数试验结果到 CSV 时出错: {e}")
else:
    print("  没有参数试验结果可供保存。")

# --- 记录主结果 (包含 SNR 提升幅度) ---
results.append({
    'filename': filename_short,
    'noise_components': ', '.join(detect_noise_components(features)),
    'search_method': SEARCH_METHOD,
    'snr_original_est': snr_original_est,          # 原始估计 SNR
    'best_estimated_snr': best_snr_est if best_snr_est > -np.inf else np.nan, # 最佳估计 SNR
    'snr_improvement_est': snr_improvement_est,    # !! 新增: 估计 SNR 提升 !!
    'snr_original_vs_denoised': snr_original_vs_denoised, # 保真度 SNR
    'best_params_found (est_SNR)': str(best_params) if best_params else "None",
    'output_path_best_estSNR': output_path_final
})

except Exception as e:
    print(f"处理文件 {noisy_path} 时发生主流程错误: {e}")
    import traceback
    traceback.print_exc()
    # 记录错误信息到主结果
    results.append({
        'filename': filename_short, 'noise_components': 'Error',
        'search_method': SEARCH_METHOD, 'snr_original_est': np.nan,
        'best_estimated_snr': np.nan,
        'snr_improvement_est': np.nan,          # !! 新增: 记录 NaN !!
        'snr_original_vs_denoised': np.nan,
        'best_params_found (est_SNR)': "Error",
        'output_path_best_estSNR': "Error"})

# --- 打印并保存主总结表格 ---
print("\n\n" + "="*80 + "\n--- 处理结果总结 ---\n" + "="*80)
if results:
    results_df = pd.DataFrame(results)
    # !! 更新主总结表格的列顺序 !!
    main_cols_order = [
        'filename', 'noise_components', 'search_method',
        'snr_original_est',          # 原始估计 SNR
        'best_estimated_snr',        # 最佳估计 SNR
        'snr_improvement_est',       # !! 新增列: 估计 SNR 提升 !!
        'snr_original_vs_denoised',  # 保真度 SNR (原始 vs 去噪)
        'best_params_found (est_SNR)', # 找到的最佳参数
        'output_path_best_estSNR'    # 输出文件路径
    ]
    main_cols_existing = [col for col in main_cols_order if col in results_df.columns]

    # 设置 Pandas 显示选项
    pd.set_option('display.float_format', '{:.2f}'.format)
    pd.set_option('display.max_columns', None)
    pd.set_option('display.width', 250)
    pd.set_option('display.max_colwidth', 60)

    print(results_df[main_cols_existing].to_string(index=False)) # 打印主总结

    # 更新主总结 CSV 文件名
    main_csv_path = os.path.join(output_dir, f"main_summary_denoising_results_v5.csv")
    try:
        results_df[main_cols_existing].to_csv(main_csv_path, index=False, float_format='%.2f')
        print(f"\n 主结果总结已保存到 CSV 文件: {main_csv_path}")
    except Exception as e:
        print(f"\n 保存主结果总结到 CSV 时出错: {e}")
else:
    print("没有文件被成功处理。")

print("\n" + "="*80 + "\n 脚本执行完毕。请检查输出文件和总结表格。 \n" + "="*80)

```